



TRƯỜNG ĐẠI HỌC QUY NHƠN

TIỂU LUẬN HỌC PHẦN
LÝ THUYẾT TỐI ƯU

THUẬT TOÁN DI TRUYỀN:
CƠ SỞ LÝ THUYẾT VÀ CÁC
ỨNG DỤNG

Trình độ đào tạo:	Thạc sĩ
Học viên thực hiện:	Nguyễn Hồ Bảo Thiên
Lớp:	Khoa học dữ liệu K28B
Giảng viên hướng dẫn:	TS. Nguyễn Văn Vũ

Gia Lai, 2026

Mục lục

1	Cơ sở lý thuyết về Thuật toán Di truyền	7
1.1	Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa	7
1.2	Cơ sở Khoa học và Nguyên lý Hoạt động của GA	9
1.3	Các Thành phần và Phép toán Cốt lõi	11
1.4	Quy trình Thuật toán và Điều kiện Dừng	16
1.5	Lập trình Di truyền	18
2	Thực nghiệm	24
2.1	Tối ưu không ràng buộc	24
2.2	Tối ưu có ràng buộc	31
2.3	Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ	38
2.4	Hồi quy Symbolic bằng Lập trình Di truyền	40
2.5	Tổng kết Chương	41
	Tài liệu tham khảo	42

Danh sách hình vẽ

1.1	Ví dụ lai ghép trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với lai ghép đơn điểm – đoạn sắp hoán đổi tô cam trên cha mẹ, đoạn vừa nhận được ở cá thể con tô xanh lá. Dưới: mã hóa số thực với lai ghép trộn theo (1.4), $\alpha = 0,75$	14
1.2	Ví dụ đột biến trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với đột biến đảo bit tại vị trí thứ 3. Dưới: mã hóa số thực với đột biến Gauss, cộng nhiễu ngẫu nhiên vào vị trí thứ 2 theo (1.5). Vị trí bị thay đổi tô cam trước đột biến; giá trị kết quả tô xanh lá sau đột biến. . . .	16
1.3	Chu trình tiến hóa của Thuật toán Di truyền	17
1.4	Lai ghép trao đổi cây con giữa hai cây cha mẹ (a)–(b), tạo ra hai cây con (c)–(d). Nút cắt tô cam; phần vừa nhận từ cha mẹ kia ở mỗi cây con tô xanh lá.	22
1.5	Ba toán tử đột biến minh họa trên cây $x + 3$ (đột biến rút gọn minh họa trên cây lớn hơn $(x + 3) \times x$). Nút hoặc nhánh bị thay đổi tô cam trên cây trước đột biến (a, c, e); nút hoặc nhánh kết quả tô xanh lá trên cây sau đột biến (b, d, f).	23
2.1	Hàm Easom: đồ thị trong không gian ba chiều và diễn biến giá trị tốt nhất qua các thế hệ. Đường nét đứt đánh dấu thế hệ mà GA chót được nghiệm tốt nhất.	25
2.2	Hàm Rastrigin: mạng lưới cực tiểu cục bộ dày đặc quanh cực tiểu toàn cục, và diễn biến giá trị tốt nhất qua các thế hệ.	26
2.3	Hàm Rosenbrock: thung lũng hẹp và cong, và diễn biến giá trị tốt nhất qua các thế hệ — GA chót kỷ lục ngay ở thế hệ 11 rồi không cải thiện thêm trong 39 thế hệ còn lại.	28
2.4	Hàm Ackley: phổ lớn ở thang toàn cục phủ gọn sóng cực tiểu cục bộ ở thang nhỏ, và diễn biến giá trị tốt nhất qua các thế hệ.	29
2.5	Hàm Schwefel: cực tiểu toàn cục nằm gần biên miền tìm kiếm, xa vùng trông có vẻ tốt quanh gốc tọa độ, và diễn biến giá trị tốt nhất qua các thế hệ.	30
2.6	Đường hội tụ của GA trên bài toán g01	33

2.7	Đường hội tụ của GA trên bài toán g06	34
2.8	Đường hội tụ của GA trên bài toán g08	35
2.9	Đường hội tụ của GA trên bài toán g11	36
2.10	Đường hội tụ của GA trên bài toán g15	37

Danh sách bảng

1.1	Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền	10
2.1	Kết quả GA trên năm hàm benchmark không ràng buộc	31
2.2	Kết quả trên bài toán g01	33
2.3	Kết quả trên bài toán g06	34
2.4	Kết quả trên bài toán g08	35
2.5	Kết quả trên bài toán g11	36
2.6	Kết quả trên bài toán g15	37
2.7	Tổng hợp kết quả GA trên năm bài toán chuẩn CEC 2006	38
2.8	Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95	39
2.9	Kết quả Hồi quy Tuyến tính và GP trên hai đại lượng vật lý phái sinh	40

Danh mục thuật ngữ

Bảng dưới đây đối chiếu các thuật ngữ tiếng Anh được sử dụng trong tiểu luận với thuật ngữ tiếng Việt tương ứng. Dạng viết tắt, nếu có, được ghi sau dấu gạch ngang. Trong phần nội dung, các thuật ngữ chỉ xuất hiện ở dạng tiếng Việt hoặc ở dạng viết tắt đã liệt kê tại đây.

Thuật ngữ tiếng Anh	Thuật ngữ tiếng Việt
Benchmark Function	Hàm chuẩn dùng để đánh giá
Binary Encoding	Mã hóa nhị phân
Bit-flip Mutation	Đột biến đảo bit
Blend Crossover — BLX- α	Lai ghép trộn
Bloat	Hiện tượng cây biểu thức phình to
Chromosome	Nhiễm sắc thể
Cycle Crossover — CX	Lai ghép theo chu trình
Elitism	Chiến lược giữ lại cá thể ưu tú
Evolution Strategies — ES	Chiến lược Tiến hóa
Evolutionary Algorithms — EA	Họ Thuật toán Tiến hóa
Evolutionary Programming — EP	Lập trình Tiến hóa
Feasible Region	Miền khả thi
Fitness Function	Hàm độ thích nghi
Gaussian Mutation	Đột biến Gauss
Gene	Gen
Generation	Thế hệ
Genetic Algorithm — GA	Thuật toán Di truyền
Genetic Programming — GP	Lập trình Di truyền
Global Optimum	Cực trị toàn cục
Gradient	Vector đạo hàm riêng
Hoist Mutation	Đột biến rút gọn
Individual	Cá thể
Insertion Mutation	Đột biến chèn
Inversion Mutation	Đột biến đảo ngược
Local Optimum	Cực trị cục bộ

Thuật ngữ tiếng Anh	Thuật ngữ tiếng Việt
Local Search	Tìm kiếm cục bộ
Mean Absolute Error — MAE	Sai số tuyệt đối trung bình
Mean Squared Error — MSE	Sai số bình phương trung bình
Memetic Algorithm	Thuật toán ghi nhớ
Multi-point Crossover	Lai ghép đa điểm
Objective Function	Hàm mục tiêu
Order Crossover — OX	Lai ghép theo thứ tự
Parsimony Pressure	Số hạng phạt theo kích thước cây
Partially Matched Crossover — PMX	Lai ghép khớp từng phần
Permutation Encoding	Mã hóa hoán vị
Point Mutation	Đột biến điểm
Population	Quần thể
Population-based Search	Tìm kiếm dựa trên quần thể
Rank-based Selection	Chọn lọc theo thứ hạng
Real-value Encoding	Mã hóa số thực
Root Mean Squared Error — RMSE	Căn bậc hai của sai số bình phương trung bình
Roulette Wheel Selection	Chọn lọc theo vòng quay Roulette
Search Space	Không gian tìm kiếm
Single-point Crossover	Lai ghép đơn điểm
Stagnation	Quần thể ngừng cải thiện
Subtree Crossover	Lai ghép trao đổi cây con
Subtree Mutation	Đột biến thay thế cây con
Survival of the Fittest	Chọn lọc tự nhiên
Swap Mutation	Đột biến hoán đổi
Tournament Selection	Chọn lọc giải đấu
Travelling Salesman Problem — TSP	Bài toán người bán hàng
Tree/Graph Encoding	Mã hóa dựa trên cấu trúc cây hoặc đồ thị
Uniform Crossover	Lai ghép đồng nhất

Chương 1

Cơ sở lý thuyết về Thuật toán Di truyền

Chương này trình bày các nền tảng lý thuyết cần thiết của Thuật toán Di truyền. Nội dung chương được tổ chức thành năm phần chính: mục đầu tiên phát biểu bài toán tối ưu hóa tổng quát và giới thiệu họ thuật toán tiến hóa; mục thứ hai trình bày cơ sở sinh học cũng như nguyên lý hoạt động của GA; mục thứ ba phân tích chi tiết các thành phần và toán tử cốt lõi; mục thứ tư tổng hợp quy trình tổng thể của thuật toán và điều kiện dừng; mục cuối cùng giới thiệu Lập trình Di truyền như một mở rộng trực tiếp của GA.

1.1 Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa

Bài toán Tối ưu hóa và Không gian Tìm kiếm

Bài toán tối ưu hóa tổng quát có thể được phát biểu như sau: tìm một vector biến quyết định $x = (x_1, x_2, \dots, x_n)$ sao cho hàm mục tiêu $f(x)$ đạt giá trị cực tiểu hoặc cực đại, trong khi x vẫn thỏa mãn một tập ràng buộc xác định miền khả thi Ω :

$$\min_{x \in \Omega} f(x) \quad \text{hoặc} \quad \max_{x \in \Omega} f(x). \quad (1.1)$$

Miền $\Omega \subseteq \mathbb{R}^n$, còn được gọi là không gian tìm kiếm, thường được xác định bởi một hệ ràng buộc đẳng thức và bất đẳng thức trên các biến quyết định. Bài toán được phân loại là **có ràng buộc** khi Ω là một tập con thực sự của không gian biến, và **không ràng buộc** khi Ω trùng với toàn bộ không gian biến.

Trong thực tiễn, tối ưu hóa thường đối mặt với hai thách thức chủ yếu như sau.

- **Vấn đề cực trị cục bộ.** Khi hàm mục tiêu không lồi hoặc đa cực trị, các thuật

toán chỉ khai thác thông tin cục bộ quanh điểm hiện tại, chẳng hạn đạo hàm, có xu hướng hội tụ về cực trị cục bộ gần với điểm khởi tạo thay vì cực trị toàn cục trên toàn miền Ω . Đây chính là hạn chế cốt lõi của các phương pháp tối ưu dựa trên gradient khi áp dụng cho bài toán không lồi.

- **Độ phức tạp tổ hợp của bài toán NP-hard.** Đối với nhiều lớp bài toán tối ưu tổ hợp, tiêu biểu như bài toán người bán hàng hay bài toán lập lịch, số lượng phương án khả thi tăng theo hàm mũ hoặc giai thừa của kích thước bài toán. Kết quả là, việc duyệt toàn bộ không gian tìm kiếm trở nên bất khả thi về mặt tính toán ngay cả với các bài toán có kích thước vừa phải.

Hai thách thức nêu trên chính là động lực thúc đẩy sự phát triển của các phương pháp tìm kiếm không dựa trên đạo hàm, đồng thời có khả năng khám phá nhiều vùng khác nhau của không gian tìm kiếm. Một trong những hướng tiếp cận tiêu biểu của nhóm phương pháp này là họ thuật toán tiến hóa, được giới thiệu trong mục tiếp theo.

Họ Thuật toán Tiến hóa

Các phương pháp tối ưu cổ điển dựa trên đạo hàm như Gradient Descent hay phương pháp Newton vận hành trên một điểm duy nhất: ở mỗi bước lặp, điểm hiện tại được dịch chuyển theo hướng mà đạo hàm chỉ ra. Cách làm này hội tụ nhanh và có cơ sở lý thuyết chặt chẽ, nhưng đòi hỏi hàm mục tiêu phải khả vi và chỉ bảo đảm tìm được cực trị cục bộ nằm gần điểm khởi tạo.

Ở một hướng tiếp cận khác, **metaheuristic** là các chiến lược tìm kiếm ở mức tổng quát, được xây dựng để làm việc trong chính những điều kiện mà phương pháp cổ điển không áp dụng được: hàm mục tiêu không khả vi, có nhiễu, không lồi, hoặc thậm chí không tồn tại biểu thức giải tích tường minh. Sự đánh đổi ở đây là rõ ràng. Metaheuristic từ bỏ bảo đảm về tính tối ưu tuyệt đối để nhận lại phạm vi áp dụng rộng hơn nhiều, và mục tiêu thực tế của nó là đạt được một nghiệm đủ tốt trong khoảng thời gian chấp nhận được.

Trong số các metaheuristic, nhóm **tìm kiếm dựa trên quần thể** duy trì đồng thời nhiều nghiệm ứng viên thay vì một nghiệm duy nhất, rồi để cả tập hợp đó cùng tiến hóa qua các thế hệ. Việc khảo sát nhiều vùng của không gian tìm kiếm cùng lúc làm giảm đáng kể nguy cơ hội tụ sớm về một cực trị cục bộ. Quan trọng hơn, các nghiệm trong quần thể có thể trao đổi thông tin với nhau để tạo ra nghiệm mới, điều mà một thuật toán chỉ theo dõi một điểm không thể thực hiện được. **Họ Thuật toán Tiến hóa** là đại diện tiêu biểu nhất của nhóm này, với quá trình tìm kiếm được dẫn dắt bởi nguyên lý chọn lọc tự nhiên của Darwin.

Mọi thuật toán thuộc họ EA đều vận hành theo một khung chung gồm năm bước: khởi tạo ngẫu nhiên một quần thể nghiệm, đánh giá chất lượng của từng nghiệm, ưu

tiên những nghiệm tốt hơn khi chọn cá thể sinh sản, sinh nghiệm mới bằng các toán tử biến đổi ngẫu nhiên, và lặp lại chu trình cho tới khi thỏa mãn điều kiện dừng. Bốn nhánh chính của họ này khác nhau chủ yếu ở cách biểu diễn nghiệm và ở toán tử biến đổi được đặt làm trọng tâm:

- **Thuật toán Di truyền:** đặt lai ghép làm toán tử tìm kiếm chính, kết hợp thông tin di truyền từ hai cá thể cha mẹ để tạo ra cá thể con. Đây là đối tượng nghiên cứu chính của tiểu luận.
- **Chiến lược Tiến hóa:** do Rechenberg và Schwefel phát triển cho bài toán tối ưu tham số thực, lấy đột biến Gauss làm toán tử chính và bổ sung cơ chế tự thích nghi biên độ đột biến trong quá trình tiến hóa.
- **Lập trình Tiến hóa:** do Fogel đề xuất, tiến hóa chủ yếu bằng đột biến trên biểu diễn hành vi của cá thể và hầu như không sử dụng lai ghép.
- **Lập trình Di truyền:** do Koza phát triển, mở rộng GA để tiến hóa trực tiếp các cấu trúc dạng cây biểu thức hoặc chương trình máy tính, thường dùng để tự động khám phá công thức toán học hoặc chương trình cho một tác vụ đặt ra từ trước.

Trong phạm vi tiểu luận này, Thuật toán Di truyền được khảo sát trên cả ba cách mã hóa nghiệm. Mã hóa số thực được dùng cho bài toán tối ưu không ràng buộc và bài toán tối ưu có ràng buộc, mã hóa hoán vị cho bài toán người du lịch, còn mã hóa dạng cây biểu thức theo nguyên lý Lập trình Di truyền được dùng cho bài toán hồi quy symbolic. Khung vận hành năm bước vừa nêu sẽ được cụ thể hóa dần ở các mục tiếp theo, lần lượt qua cách mã hóa nghiệm, hàm độ thích nghi, và ba nhóm toán tử chọn lọc, lai ghép và đột biến; riêng nhánh Lập trình Di truyền được trình bày tách riêng ở Mục 1.5.

1.2 Cơ sở Khoa học và Nguyên lý Hoạt động của GA

Nguồn gốc Lịch sử và Nguyên lý Sinh học

Thuật toán Di truyền do John Holland đề xuất năm 1975 tại Đại học Michigan trong công trình *Adaptation in Natural and Artificial Systems* [1], và sau đó được David Goldberg hệ thống hóa cũng như phổ biến rộng rãi hơn thông qua cuốn sách năm 1989 *Genetic Algorithms in Search, Optimization, and Machine Learning* [2].

Ý tưởng nền tảng của GA là mô phỏng ba nguyên lý cốt lõi trong học thuyết tiến hóa của Darwin:

- **Chọn lọc tự nhiên:** những cá thể có khả năng thích nghi tốt hơn với môi trường sẽ có xác suất sống sót và sinh sản cao hơn.
- **Di truyền:** các đặc điểm của cá thể cha mẹ được truyền lại cho thế hệ con thông qua vật chất di truyền.
- **Biến dị:** các sai khác ngẫu nhiên giữa các cá thể, hình thành thông qua lai ghép và đột biến, tạo nên tính đa dạng di truyền – điều kiện tiên quyết để quá trình chọn lọc có tác dụng.

Dựa trên ba nguyên lý nêu trên, GA thiết lập một phép tương ứng giữa các khái niệm sinh học và các thành phần thuật toán. Phép tương ứng này được tóm tắt trong Bảng 1.1.

Bảng 1.1: Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền

Sinh học	Thuật ngữ tiếng Anh	Thuật toán Di truyền
Nhiễm sắc thể	Chromosome	Mã hóa nghiệm
Gen	Gene	Một biến thành phần trong nghiệm
Cá thể	Individual	Một nghiệm ứng viên
Quần thể	Population	Tập hợp các nghiệm tại một thế hệ
Độ thích nghi	Fitness	Giá trị hàm độ thích nghi
Thế hệ	Generation	Một vòng lặp tiến hóa

Nhờ phép tương ứng này, một bài toán tối ưu bất kỳ có thể được diễn dịch thành một quá trình “tiến hóa nhân tạo”. Cụ thể, thuật toán xuất phát từ một quần thể các nghiệm ngẫu nhiên; trong suốt quá trình tiến hóa, các nghiệm có chất lượng tốt hơn sẽ có nhiều cơ hội được chọn làm cha mẹ hơn. Qua nhiều thế hệ, quần thể dần dịch chuyển về phía những vùng của không gian tìm kiếm có giá trị hàm mục tiêu tốt.

Phương pháp Mã hóa Nghiệm

Việc áp dụng GA cho một bài toán cụ thể bắt đầu bằng bước mã hóa: mỗi nghiệm cần được biểu diễn dưới dạng một “nhiễm sắc thể” mà các toán tử di truyền có thể thao tác trực tiếp. Bốn cách mã hóa phổ biến nhất được trình bày như sau.

- **Mã hóa nhị phân:** nghiệm được biểu diễn bằng một chuỗi bit gồm 0 và 1, chẳng hạn 10110010. Đây là cách mã hóa nguyên thủy nhất của GA cổ điển do Holland đề xuất, phù hợp với các bài toán rời rạc hoặc logic. Tuy nhiên, khi áp dụng cho biến số thực, cách mã hóa này đòi hỏi một bước rời rạc hóa, làm giảm độ chính xác của nghiệm.

- **Mã hóa số thực:** nghiệm là một dãy số thực, chẳng hạn $[3.14, -0.05, 12.8]$, trong đó mỗi phần tử tương ứng trực tiếp với một biến quyết định. Cách mã hóa này phù hợp một cách tự nhiên với các bài toán tối ưu liên tục nhiều biến và loại trừ được sai số lượng tử hóa của mã hóa nhị phân. Đây cũng là cách mã hóa được sử dụng trong phần cài đặt thực nghiệm của tiểu luận tại Chương 2.
- **Mã hóa hoán vị:** nghiệm là một thứ tự sắp xếp của một tập hợp phần tử, chẳng hạn $[3, 1, 4, 2, 5]$. Đây là cách mã hóa tự nhiên cho các bài toán định tuyến và lập lịch như TSP hay bài toán phân công công việc, nơi nghiệm về bản chất là một hoán vị.
- **Mã hóa dựa trên cấu trúc:** nghiệm được biểu diễn dưới dạng cây biểu thức hoặc đồ thị, chủ yếu được sử dụng trong Lập trình Di truyền nhằm tiến hóa các công thức hoặc chương trình.

Cần lưu ý rằng việc lựa chọn cách mã hóa quyết định trực tiếp đến loại toán tử lai ghép và đột biến có thể áp dụng. Vấn đề này sẽ được phân tích chi tiết trong Mục 1.3.

1.3 Các Thành phần và Phép toán Cốt lõi

Hàm độ thích nghi và Ràng buộc

Hàm độ thích nghi đóng vai trò đánh giá chất lượng của từng cá thể trong quần thể, và là cơ sở cho toàn bộ quá trình chọn lọc. Đối với bài toán không ràng buộc, hàm thích nghi thường được xây dựng trực tiếp từ hàm mục tiêu; chẳng hạn, đối với bài toán cực tiểu hóa, ta có thể đảo dấu hàm mục tiêu do GA nguyên thủy được phát biểu dưới dạng cực đại hóa độ thích nghi.

Đối với bài toán **có ràng buộc**, trọng tâm của tiểu luận này, cách tiếp cận kinh điển là đưa ràng buộc vào hàm mục tiêu thông qua một số hạng phạt, tạo thành hàm thích nghi đã phạt:

$$F(x) = f(x) \pm \sum_i \lambda_i \cdot g_i(x), \quad (1.2)$$

trong đó $g_i(x)$ đo mức độ vi phạm ràng buộc thứ i (nhận giá trị 0 nếu ràng buộc được thỏa mãn), và $\lambda_i > 0$ là hệ số phạt tương ứng. Dấu của số hạng phạt cũng như độ lớn của λ_i được chọn tùy theo bài toán là cực tiểu hay cực đại hóa, sao cho một cá thể vi phạm ràng buộc luôn bị đánh giá thấp hơn một cá thể khả thi có cùng giá trị hàm mục tiêu.

Hàm phạt tồn tại một hạn chế cố hữu: hệ số λ_i phải được xác định thủ công và giữ cố định trong suốt quá trình tiến hóa. Nếu chọn λ_i quá nhỏ, quần thể có xu hướng bỏ qua các ràng buộc; ngược lại, nếu chọn λ_i quá lớn, số hạng phạt sẽ lấn át hoàn toàn hàm mục tiêu, khiến quần thể chỉ tập trung thỏa mãn ràng buộc mà không

còn khả năng phân biệt giữa các nghiệm khả thi tốt và kém. Để khắc phục hạn chế này, nhiều kỹ thuật xử lý ràng buộc nâng cao đã được đề xuất, không còn dựa vào một hệ số phạt cố định.

Quy tắc khả thi của Deb thay hàm phạt bằng một quy tắc so sánh trực tiếp giữa hai cá thể, dựa trên ba trường hợp: (i) giữa hai cá thể đều khả thi, cá thể có giá trị hàm mục tiêu tốt hơn luôn được ưu tiên; (ii) giữa một cá thể khả thi và một cá thể không khả thi, cá thể khả thi luôn được ưu tiên bất kể giá trị hàm mục tiêu; (iii) giữa hai cá thể đều không khả thi, cá thể có tổng mức vi phạm ràng buộc nhỏ hơn được ưu tiên. Quy tắc này loại bỏ hoàn toàn nhu cầu xác định hệ số phạt, vì việc so sánh không còn cộng gộp mức vi phạm vào hàm mục tiêu.

Ngưỡng ε giảm dần do Takahama và Sato đề xuất không yêu cầu mức vi phạm bằng 0 ngay từ đầu. Thay vào đó, cá thể được xem là khả thi khi tổng mức vi phạm không vượt quá ngưỡng $\varepsilon(t)$. Ngưỡng này bắt đầu ở giá trị dương, cho phép quần thể khám phá cả vùng không khả thi, rồi giảm dần về 0 tại thế hệ T_c , từ đó hướng quần thể hội tụ vào miền khả thi.

Các Phép chọn lọc

Toán tử chọn lọc quyết định những cá thể nào của quần thể hiện tại sẽ được chọn làm cha mẹ để sinh ra thế hệ tiếp theo. Nguyên tắc chung của toán tử này là thiên vị có hướng về phía các cá thể có độ thích nghi cao. Ba phương pháp chọn lọc phổ biến nhất được trình bày dưới đây.

- **Vòng quay Roulette:** xác suất chọn cá thể i tỉ lệ thuận với độ thích nghi của nó so với tổng độ thích nghi của toàn quần thể:

$$P_i = \frac{f_i}{\sum_{j=1}^N f_j}. \quad (1.3)$$

Hạn chế của phương pháp này là khi một cá thể có độ thích nghi vượt trội hẳn so với phần còn lại, cá thể đó sẽ chiếm phần lớn xác suất được chọn, từ đó làm quần thể nhanh chóng mất đi tính đa dạng.

- **Giải đấu:** chọn ngẫu nhiên k cá thể từ quần thể, sau đó chọn cá thể tốt nhất trong nhóm làm cha mẹ. Tham số k , gọi là kích thước giải đấu, kiểm soát trực tiếp áp lực chọn lọc: k càng lớn thì áp lực chọn lọc càng cao, quần thể hội tụ càng nhanh, nhưng đổi lại rủi ro mất đa dạng cũng tăng theo.
- **Chọn lọc theo thứ hạng:** cá thể được chọn dựa trên thứ hạng của nó trong quần thể đã được sắp xếp theo độ thích nghi, thay vì dựa trên giá trị thích nghi tuyệt đối. Cách tiếp cận này khắc phục được hiện tượng một cá thể vượt trội áp đảo toàn bộ xác suất chọn như trong phương pháp vòng quay Roulette.

Bên cạnh các toán tử chọn lọc cha mẹ nêu trên, chiến lược lựa chọn cá thể chuyển sang thế hệ sau còn bao gồm cơ chế giữ lại cá thể ưu tú, được trình bày riêng trong Mục 1.3 do vai trò và cách áp dụng của nó khác biệt so với chọn lọc cha mẹ thông thường.

Các Phép lai ghép

Toán tử lai ghép kết hợp vật chất di truyền của hai cá thể cha mẹ để tạo ra cá thể con mới. Tần suất lai ghép p_c là xác suất một cặp cha mẹ đã chọn thực sự tham gia lai ghép; phần còn lại được sao chép nguyên vẹn sang thế hệ con. p_c thường được đặt ở mức cao để đa số cặp cha mẹ tham gia lai ghép, giá trị cụ thể tùy vào bài toán và được xác định bằng thực nghiệm.

Đối với mã hóa nhị phân và số thực, ba toán tử lai ghép phổ biến nhất là:

- **Lai ghép đơn điểm:** chọn ngẫu nhiên một vị trí cắt trên chuỗi nhiễm sắc thể. Hai cá thể con được tạo ra bằng cách hoán đổi các đoạn nằm sau vị trí cắt giữa hai cha mẹ.
- **Lai ghép đa điểm:** là dạng tổng quát của lai ghép đơn điểm với nhiều vị trí cắt; các đoạn xen kẽ giữa hai cha mẹ được hoán đổi cho nhau.
- **Lai ghép đồng nhất:** giá trị của mỗi gen tại vị trí con được quyết định độc lập và ngẫu nhiên, lấy từ cha hoặc mẹ, không phụ thuộc vào vị trí của gen trong chuỗi.

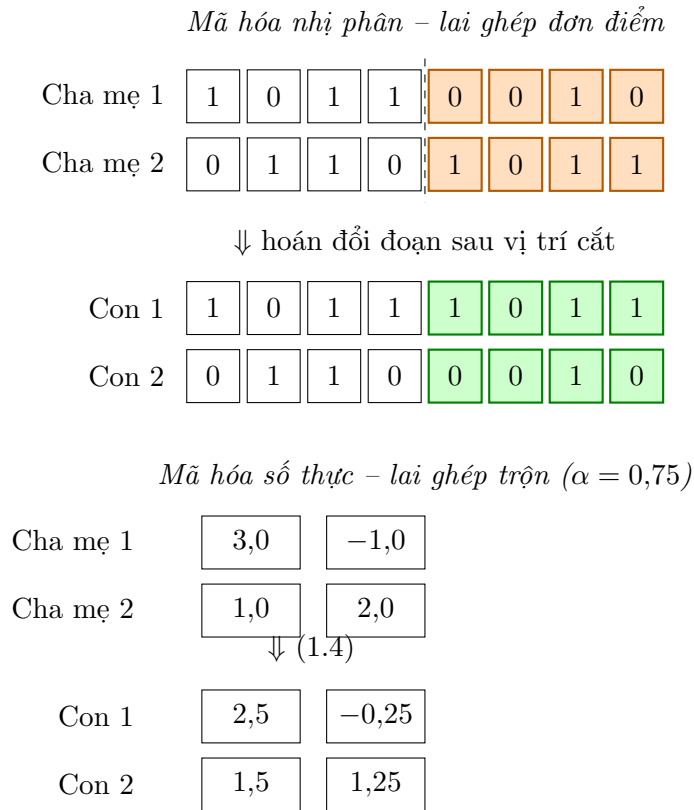
Đối với mã hóa số thực, một họ toán tử quan trọng khác là lai ghép trộn, cũng chính là toán tử được sử dụng trong phần cài đặt của tiểu luận. Xét hai cá thể cha mẹ $p_1, p_2 \in \mathbb{R}^n$ và hệ số trộn α được lấy mẫu ngẫu nhiên, có thể nhận giá trị nằm ngoài khoảng $[0, 1]$, hai cá thể con được tạo ra theo tổ hợp tuyến tính:

$$c_1 = \alpha \cdot p_1 + (1 - \alpha) \cdot p_2, \quad c_2 = \alpha \cdot p_2 + (1 - \alpha) \cdot p_1. \quad (1.4)$$

Khi α được lấy mẫu từ một khoảng mở rộng ra ngoài $[0, 1]$, chẳng hạn $\alpha \sim \mathcal{U}(-0.25, 1.25)$, cá thể con có thể nằm ngoài đoạn thẳng nối p_1 và p_2 . Nhờ vậy, toán tử này cân bằng được giữa khả năng khai thác (khai thác vùng lân cận cha mẹ) và khả năng khám phá (tiếp cận các vùng nằm ngoài đoạn nối hai cha mẹ), thay vì chỉ co dần khoảng tìm kiếm qua các thế hệ.

Hình 1.1 minh họa lai ghép đơn điểm trên mã hóa nhị phân và lai ghép trộn trên mã hóa số thực – hai cách mã hóa được nhắc đến nhiều nhất trong chương này, lần lượt là cách mã hóa cổ điển của GA và cách mã hóa dùng trong phần cài đặt thực nghiệm của tiểu luận. Với mã hóa nhị phân, vị trí cắt được chọn ngẫu nhiên sau gen thứ 4; đoạn từ vị trí 5 đến 8 (tô cam) được hoán đổi giữa hai cha mẹ để tạo ra hai cá

thể con, với đoạn vừa nhận tô xanh lá. Với mã hóa số thực, hai cá thể con được tính trực tiếp theo công thức lai ghép trộn (1.4) với $\alpha = 0,75$.



Hình 1.1: Ví dụ lai ghép trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với lai ghép đơn điểm – đoạn sắp hoán đổi tô cam trên cha mẹ, đoạn vừa nhận được ở cá thể con tô xanh lá. Dưới: mã hóa số thực với lai ghép trộn theo (1.4), $\alpha = 0,75$.

Đối với mã hóa hoán vị trong các bài toán định tuyến và lập lịch, việc hoán đổi trực tiếp từng đoạn như các toán tử trên có thể tạo ra cá thể con vi phạm tính hoán vị do có phần tử bị lặp lại hoặc thiếu sót. Vì vậy, các toán tử lai ghép chuyên biệt cần được sử dụng:

- **Lai ghép khớp từng phần:** hoán đổi một đoạn con giữa hai cha mẹ theo cách tương tự lai ghép đơn điểm, sau đó điều chỉnh các vị trí còn lại dựa trên ánh xạ tương ứng từng phần nhằm đảm bảo tính hoán vị của cá thể con.
- **Lai ghép theo thứ tự:** giữ nguyên một đoạn con từ cha mẹ thứ nhất, các phần tử còn lại được điền vào theo đúng thứ tự xuất hiện của chúng trong cha mẹ thứ hai.
- **Lai ghép theo chu trình:** xác định các “chu trình” giữa vị trí phần tử của hai cha mẹ; mỗi cá thể con được tạo ra bằng cách giữ nguyên các chu trình xen kẽ lấy từ mỗi cha mẹ.

Các Phép đột biến

Toán tử đột biến tạo ra biến dị ngẫu nhiên trên một cá thể. Tần suất đột biến p_m trên mỗi gen quyết định mức độ biến dị đưa vào quần thể ở mỗi thế hệ; giá trị cụ thể phụ thuộc vào cách mã hóa và bài toán, được xác định bằng thực nghiệm. Vai trò chính của đột biến là duy trì đa dạng di truyền cho quần thể, đồng thời giúp thuật toán thoát ra khỏi những vùng mà toán tử lai ghép một mình không thể tiếp cận được. Một số dạng đột biến tiêu biểu bao gồm:

- **Đột biến đảo bit:** áp dụng cho mã hóa nhị phân; mỗi bit được đảo giá trị ($0 \leftrightarrow 1$) độc lập với xác suất p_m .
- **Đột biến Gauss:** áp dụng cho mã hóa số thực; mỗi biến x_i được cộng thêm một nhiễu ngẫu nhiên, phổ biến nhất là nhiễu Gauss:

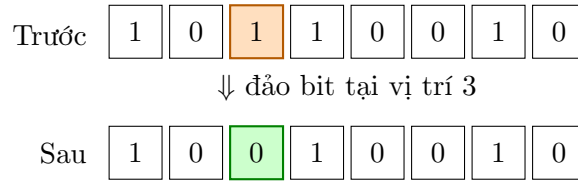
$$x'_i = x_i + \mathcal{N}(0, \sigma_i^2), \quad (1.5)$$

trong đó độ lệch chuẩn σ_i thường được đặt tỉ lệ với độ rộng miền giá trị của biến x_i , nhằm đảm bảo bước đột biến có ý nghĩa tương đương giữa các biến có thang đo khác nhau.

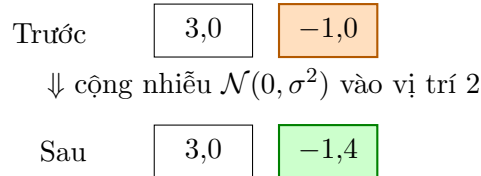
- **Đột biến hoán đổi, đảo ngược và chèn:** áp dụng cho mã hóa hoán vị; lần lượt là hoán đổi vị trí hai phần tử, đảo ngược thứ tự một đoạn con, và di chuyển một phần tử sang vị trí khác trong chuỗi. Cả ba thao tác đều bảo toàn tính hoán vị của cá thể.

Tương tự, Hình 1.2 minh họa đột biến đảo bit trên mã hóa nhị phân và đột biến Gauss trên mã hóa số thực. Với mã hóa nhị phân, gen tại vị trí 3 (tô cam) được đảo giá trị từ 1 thành 0. Với mã hóa số thực, giá trị tại vị trí 2 (tô cam) được cộng thêm một nhiễu Gauss theo (1.5), ở đây lấy mẫu được $\mathcal{N}(0, \sigma^2) = -0,4$, cho ra giá trị mới $-1,4$ (tô xanh lá).

Mã hóa nhị phân – đột biến đảo bit



Mã hóa số thực – đột biến Gauss



Hình 1.2: Ví dụ đột biến trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với đột biến đảo bit tại vị trí thứ 3. Dưới: mã hóa số thực với đột biến Gauss, cộng nhiễu ngẫu nhiên vào vị trí thứ 2 theo (1.5). Vị trí bị thay đổi tô cam trước đột biến; giá trị kết quả tô xanh lá sau đột biến.

Chiến lược giữ lại Cá thể Ưu tú

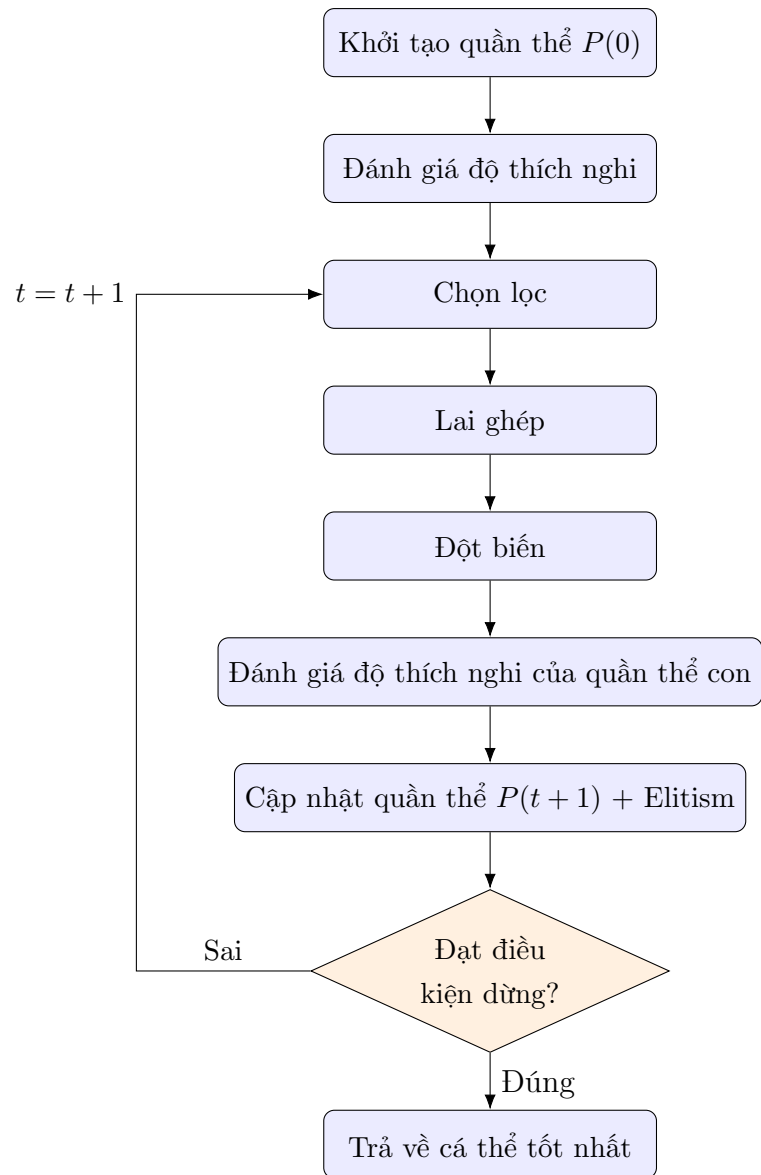
Do các toán tử chọn lọc, lai ghép và đột biến đều mang tính ngẫu nhiên, không có cơ chế nào đảm bảo cá thể tốt nhất của thế hệ hiện tại sẽ được duy trì sang thế hệ kế tiếp. Trên lý thuyết, độ thích nghi tốt nhất của toàn quần thể hoàn toàn có thể giảm đi giữa hai thế hệ liên tiếp. Chiến lược **Elitism** ra đời nhằm khắc phục vấn đề này: k cá thể tốt nhất của thế hệ hiện tại được giữ nguyên vẹn và chuyển trực tiếp sang thế hệ sau mà không trải qua lai ghép hay đột biến. Nhờ đó, độ thích nghi tốt nhất từng đạt được sẽ không bao giờ giảm qua các thế hệ.

Việc lựa chọn kích thước elitism k đòi hỏi một sự cân nhắc. Nếu k quá nhỏ, các cá thể tốt vẫn có thể bị mất do tác động của các toán tử ngẫu nhiên. Ngược lại, nếu k quá lớn, áp lực khám phá của quần thể sẽ giảm sút, làm suy giảm tính đa dạng và khiến quá trình tìm kiếm hội tụ sớm về một vùng hẹp của không gian tìm kiếm.

1.4 Quy trình Thuật toán và Điều kiện Dừng

Sơ đồ khối và Mã giả

Phối hợp các thành phần đã trình bày trong Mục 1.3, một thể hệ của GA điển hình bao gồm chu trình lặp lại sau: chọn lọc cha mẹ, lai ghép, đột biến, đánh giá quần thể con, rồi cập nhật quần thể mới với cơ chế elitism. Chu trình này được minh họa tổng quát trong Hình 1.3.



Hình 1.3: Chu trình tiến hóa của Thuật toán Di truyền

Để mô tả chính xác hơn, Thuật toán 1 trình bày chu trình tiến hóa trên dưới dạng mã giả (pseudocode).

Thuật toán 1 Mã giả tổng quát của Thuật toán Di truyền

```
1:  $t \leftarrow 0$ 
2: Khởi tạo quần thể ban đầu  $P(0)$  ngẫu nhiên
3: Đánh giá độ thích nghi của  $P(0)$ 
4: while điều kiện dừng chưa thỏa mãn do
5:    $t \leftarrow t + 1$ 
6:   Chọn lọc tập cha mẹ  $S(t)$  từ  $P(t - 1)$ 
7:   Thực hiện lai ghép trên  $S(t)$ , tạo tập con  $C(t)$ 
8:   Thực hiện đột biến trên  $C(t)$ 
9:   Đánh giá độ thích nghi của  $C(t)$ 
10:   $P(t) \leftarrow$  cập nhật quần thể mới (áp dụng elitism)
11: end while
12: return cá thể tốt nhất tìm được
```

Tiêu chuẩn Dừng và Đánh giá sự Hội tụ

Một thuật toán tiến hóa cần được trang bị một điều kiện dừng tường minh để kết thúc quá trình lặp. Ba tiêu chuẩn dừng phổ biến, thường được kết hợp sử dụng với nhau, được trình bày như sau.

- **Số thế hệ tối đa G_{max}** : đây là tiêu chuẩn đơn giản nhất, đồng thời giúp kiểm soát trực tiếp ngân sách tính toán. Tuy nhiên, việc lựa chọn G_{max} cần cân nhắc: nếu G_{max} quá nhỏ, thuật toán có thể dừng lại trước khi quần thể hội tụ; nếu G_{max} quá lớn, tài nguyên tính toán sẽ bị lãng phí sau khi quần thể đã hội tụ.
- **Ngưỡng độ thích nghi mục tiêu**: thuật toán dừng ngay khi tìm được một cá thể có độ thích nghi đạt hoặc vượt ngưỡng đặt trước. Tiêu chuẩn này phù hợp khi giá trị tối ưu cần đạt đã biết trước hoặc có thể ước lượng được.
- **Độ trễ hội tụ**: thuật toán dừng khi độ thích nghi của cá thể tốt nhất không cải thiện sau K thế hệ liên tiếp, được xem là dấu hiệu cho thấy các toán tử di truyền không còn tạo ra tiến bộ đáng kể. Tuy vậy, cần lưu ý rằng tiêu chuẩn này có thể khiến thuật toán dừng quá sớm nếu K quá nhỏ, do quần thể hoàn toàn có thể trải qua giai đoạn không cải thiện tạm thời trước khi tiếp tục tiến triển ở các thế hệ sau.

1.5 Lập trình Di truyền

Lập trình Di truyền do John Koza phát triển từ cuối thập niên 1980 và được hệ thống hóa trong công trình năm 1992 [6]. GP là một mở rộng trực tiếp của GA, tương ứng

với dạng *mã hóa dựa trên cấu trúc* đã được giới thiệu trong Mục 1.2. Khác biệt cốt lõi so với GA dùng mã hóa số thực hay hoán vị là: thay vì tiến hóa một chuỗi giá trị có độ dài cố định trong khuôn dạng nghiệm đã cho trước, GP tiến hóa trực tiếp **cấu trúc** của một biểu thức hoặc chương trình. Nhờ đó, thuật toán có khả năng tự khám phá đồng thời cả hình dạng lẫn tham số của nghiệm – đây chính là lý do GP được xem là bước chuyển từ “tối ưu tham số” sang “tự động tổng hợp chương trình”. Một tính chất đáng chú ý của GP là nghiệm cuối cùng có dạng *biểu thức tường minh*, có thể đọc, phân tích và giải thích được bởi con người, khác biệt căn bản với các mô hình hộp đen như mạng nơ-ron.

Mã hóa dạng cây biểu thức

Mỗi cá thể trong GP là một cây biểu thức. Mỗi nút trong của cây là một toán tử được chọn từ một *tập toán tử* cho trước, chẳng hạn các phép toán số học $\{+, -, \times, \div\}$ hoặc hàm lượng giác $\{\sin, \cos\}$; mỗi nút lá là một biến đầu vào hoặc hằng số được chọn từ một *tập biến và hằng số* cho trước. Khi duyệt cây theo đúng cấu trúc phân cấp của nó, ta thu được một biểu thức toán học tường minh, có thể tính giá trị trên bất kỳ bộ dữ liệu đầu vào nào. Điều này cho phép toàn bộ quần thể các cây với hình dạng và nội dung khác nhau được đánh giá, so sánh một cách thống nhất như trong GA thông thường.

Việc thiết kế hai tập hợp này cần thỏa mãn hai tính chất cơ bản:

- **Tính khép kín** (closure): mọi toán tử phải chấp nhận bất kỳ giá trị nào do một toán tử khác hoặc một nút lá trả về, và luôn cho ra một kết quả hợp lệ. Ràng buộc này nảy sinh vì lai ghép và đột biến ghép các toán tử với nhau một cách ngẫu nhiên, không theo trật tự cố định nào. Chẳng hạn, nếu $\div$ xuất hiện tại một nút mà cây con ở vị trí mẫu số vừa tính ra 0 – một tình huống hoàn toàn có thể xảy ra – phép chia thông thường không xác định, khiến cả cây không tính được giá trị và cá thể đó trở nên vô nghĩa. Để tránh điều này, $\div$ thường được thay bằng *phép chia bảo vệ*: trả về 0 (hoặc một hằng số lớn) khi mẫu số bằng 0, thay vì để phép tính thất bại. Các hàm có miền xác định hẹp hơn tập số thực như $\log$ hay $\sqrt{\cdot}$ cũng cần phiên bản bảo vệ tương tự, chẳng hạn $\log(|x|)$ thay cho $\log(x)$, để luôn nhận được mọi giá trị đầu vào có thể xuất hiện trong cây.
- **Tính đủ** (sufficiency): hai tập hợp này phải chứa đủ các thành phần để biểu diễn được (ít nhất xấp xỉ) nghiệm của bài toán. Nếu dữ liệu thực sự mang quan hệ lượng giác mà tập toán tử chỉ gồm bốn phép tính số học, GP sẽ phải xấp xỉ hàm đích bằng đa thức với độ chính xác thấp.

Độ sâu tối đa của cây cũng là một tham số thiết kế quan trọng: giới hạn quá chặt sẽ thu hẹp không gian khám phá, trong khi không giới hạn đủ mức dễ dẫn đến các cây

có kích thước rất lớn, tốn kém khi đánh giá và khó diễn giải.

Khởi tạo quần thể. Khác với GA nơi quần thể ban đầu chỉ là các chuỗi ngẫu nhiên, việc sinh cây ngẫu nhiên trong GP có ba chiến lược kinh điển, tất cả đều giới hạn độ sâu tối đa $d_{\max}$:

- **Full:** mọi nhánh của cây đều có độ sâu đúng bằng $d_{\max}$, tạo ra các cây cân đối, dày đặc;
- **Grow:** mỗi nhánh có thể kết thúc ở độ sâu bất kỳ không vượt quá $d_{\max}$, tạo ra các cây không đồng nhất về hình dạng;
- **Ramped half-and-half:** kết hợp hai phương pháp trên theo tỷ lệ 1 : 1, đồng thời phân bố độ sâu tối đa của từng cây đều trên khoảng $[2, d_{\max}]$. Đây là chiến lược được khuyến nghị và sử dụng phổ biến nhất, vì nó tạo ra quần thể ban đầu đa dạng cả về kích thước lẫn hình dạng, giảm nguy cơ quần thể bị chi phối bởi một kiểu cấu trúc duy nhất ngay từ đầu.

Hàm độ thích nghi

Ứng dụng tiêu biểu nhất của GP là bài toán **symbolic regression**: tự động tìm một biểu thức toán học mô tả tốt mối quan hệ giữa tập dữ liệu quan sát (X, y) mà không giả định trước dạng hàm. Bài toán này khác biệt căn bản so với hồi quy tuyến tính hay hồi quy đa thức, nơi dạng hàm đã được ấn định sẵn và công việc còn lại chỉ là tối ưu các hệ số. Trong GP, độ thích nghi của một cây thường được xây dựng từ sai số dự đoán, tiêu biểu là sai số bình phương trung bình giữa giá trị cây tính được và giá trị y thực tế. Một số công trình còn dùng RMSE, MAE, hoặc hệ số xác định R^2 ; lựa chọn này ảnh hưởng trực tiếp đến độ nhạy của chọn lọc đối với các ngoại lai trong dữ liệu.

Một vấn đề đặc trưng của GP là hiện tượng **bloat**: kích thước cây có xu hướng tăng trưởng liên tục qua các thế hệ mà không mang lại cải thiện tương xứng về độ chính xác. Nguyên nhân được lý giải ở mức lý thuyết là các cây con trung tính (ví dụ đoạn $+0, \times 1$) tích lũy dần vì chúng không làm giảm độ thích nghi nhưng lại được bảo vệ khỏi đột biến có hại. Bloat làm tăng chi phí đánh giá, giảm khả năng diễn giải nghiệm, và có thể dẫn đến quá khớp. Để đối phó, người ta thường cộng thêm vào hàm thích nghi một **số hạng phạt** tỉ lệ với kích thước cây:

$$\text{fitness}(T) = \text{MSE}(T) + \lambda \cdot |T|, \quad (1.6)$$

trong đó $|T|$ là số nút của cây T và $\lambda > 0$ điều chỉnh mức độ ưu tiên sự gọn nhẹ so với độ chính xác. Các biện pháp bổ sung gồm giới hạn độ sâu/số nút tuyệt đối, loại bỏ cá thể vi phạm ngay sau khi sinh, và các toán tử đột biến có chủ đích rút gọn cây.

Toán tử tiến hóa

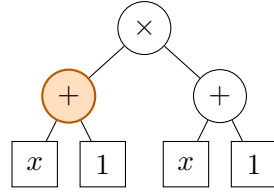
Các toán tử lai ghép và đột biến trong GP thao tác trực tiếp trên cấu trúc cây thay vì trên chuỗi có độ dài cố định:

- **Lai ghép trao đổi cây con:** chọn ngẫu nhiên một nút trên mỗi cây cha mẹ, sau đó hoán đổi hai cây con bắt rễ tại các nút đó cho nhau để tạo ra hai cây con mới. Các biến thể gồm lai ghép bất đối xứng (cây con thứ nhất lấy từ nút trong, cây con thứ hai từ nút lá) và lai ghép tự thân (cắt và dán trên cùng một cây), nhằm cân bằng giữa khám phá và khai thác;
- **Đột biến thay thế cây con:** chọn ngẫu nhiên một nút trên cây, rồi thay toàn bộ cây con bắt rễ tại nút đó bằng một cây con mới được sinh ngẫu nhiên;
- **Đột biến điểm** (point mutation): thay một nút hàm bằng một hàm khác cùng số ngôi, hoặc thay một nút lá bằng biến/hằng số khác, giữ nguyên cấu trúc cây;
- **Đột biến rút gọn** (hoist mutation): thay toàn bộ cây bằng một cây con ngẫu nhiên bên trong nó, qua đó thu nhỏ cây một cách có kiểm soát – biến thể này thường được dùng kết hợp với số hạng phạt (1.6) để chống bloat.

Các thành phần còn lại của chu trình tiến hóa, bao gồm khởi tạo quần thể, chọn lọc, elitism và điều kiện dừng, về bản chất tương tự như GA đã trình bày trong các mục trước; khác biệt duy nhất nằm ở đối tượng thao tác là **cây** thay vì **vector** số hoặc hoán vị. Chọn lọc giải đấu được ưa chuộng trong GP vì đơn giản, ít nhạy với thang đo của hàm thích nghi và dễ song song hóa khi kích thước quần thể lớn; kích thước giải đấu cụ thể tùy vào bài toán và được xác định bằng thực nghiệm. Điều kiện dừng thường là số thế hệ tối đa kết hợp với ngưỡng sai số chấp nhận được; một số hệ thống còn theo dõi độ đa dạng của quần thể để kết thúc sớm khi quần thể đã hội tụ.

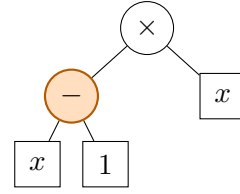
Để minh họa cụ thể cách các toán tử trên vận hành trên cây biểu thức, xét hai cây cha mẹ với tập toán tử $\{+, -, \times\}$. Cha mẹ thứ nhất là cây $(x + 1)^2$; do tập toán tử không có phép lũy thừa, cây này được biểu diễn dưới dạng $\times[+[x, 1], +[x, 1]]$. Cha mẹ thứ hai là cây $(x - 1)x$, biểu diễn $\times[-[x, 1], x]$.

Hình 1.4 minh họa phép lai ghép trao đổi cây con giữa hai cha mẹ này. Nút cắt được chọn là nhánh $+[x, 1]$ của cha mẹ thứ nhất và nhánh $-[x, 1]$ của cha mẹ thứ hai – hai nhánh này được tô màu cam trong hình. Hoán đổi hai nhánh cho nhau sinh ra hai cá thể con: $(x - 1)(x + 1) = x^2 - 1$ và $(x + 1)x = x^2 + x$, với phần vừa được ghép sang từ cha mẹ kia tô màu xanh lá. Cả hai cá thể con đều tổ hợp đúng nhân tử tuyến tính của cha mẹ này với phần còn lại của cha mẹ kia, minh họa trực tiếp cách lai ghép cây con tái tổ hợp các nhánh cây giữa nhiều cha mẹ để tạo ra cá thể mới.



(a) Cha mẹ 1

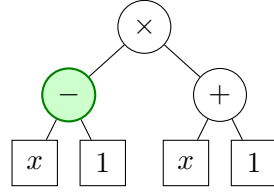
$$(x+1)^2$$



(b) Cha mẹ 2

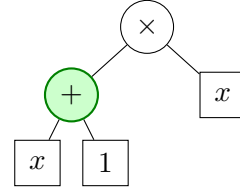
$$(x-1)x$$

⇓ Lai ghép trao đổi cây con



(c) Con 1

$$(x-1)(x+1) = x^2 - 1$$

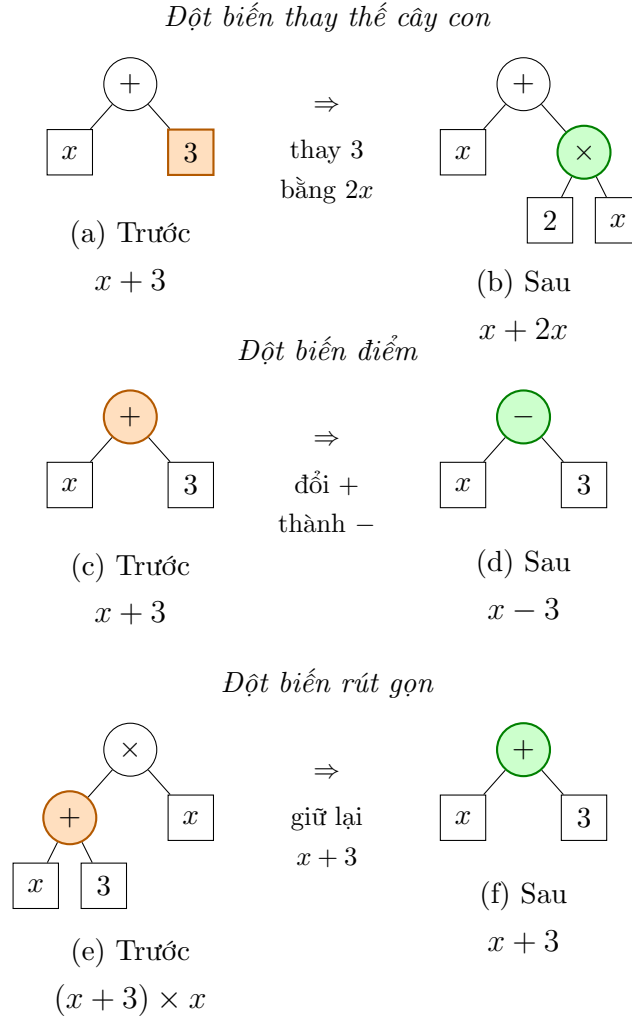


(d) Con 2

$$(x+1)x = x^2 + x$$

Hình 1.4: Lai ghép trao đổi cây con giữa hai cây cha mẹ (a)–(b), tạo ra hai cây con (c)–(d). Nút cắt tô cam; phần vừa nhận từ cha mẹ kia ở mỗi cây con tô xanh lá.

Ba toán tử đột biến cũng minh họa được trên một cây đơn giản, chẳng hạn cây $x+3$, như trong Hình 1.5. Đột biến thay thế cây con chọn một nhánh – ví dụ nhánh $3-$ và thay bằng một cây con mới sinh ngẫu nhiên, chẳng hạn $2x$, được cá thể $x+2x$. Đột biến điểm giữ nguyên cấu trúc cây, chỉ đổi một nút hàm sang hàm khác cùng số ngôi: đổi $+$ thành $-$, được cây $x-3$. Đột biến rút gọn thao tác trên một cây lớn hơn, chẳng hạn $(x+3) \times x$: chọn cây con $x+3$ bên trong nó rồi thay thế toàn bộ cá thể bằng đúng cây con đó, được cá thể mới chỉ còn $x+3$, nhỏ hơn hẳn cây gốc.



Hình 1.5: Ba toán tử đột biến minh họa trên cây $x + 3$ (đột biến rút gọn minh họa trên cây lớn hơn $(x + 3) \times x$). Nút hoặc nhánh bị thay đổi tô cam trên cây trước đột biến (a, c, e); nút hoặc nhánh kết quả tô xanh lá trên cây sau đột biến (b, d, f).

Tóm lại, chương này đã trình bày đầy đủ các nền tảng lý thuyết của GA và GP. Trên cơ sở đó, Chương 2 sẽ trình bày việc áp dụng GA cho ba nhóm bài toán đầu tiên – tối ưu không ràng buộc, tối ưu có ràng buộc, và bài toán người du lịch – cùng với việc áp dụng GP cho bài toán hồi quy symbolic trên dữ liệu gió mặt trời thực đo từ vệ tinh WIND của NASA, dự đoán độ lớn từ trường và vận tốc Alfvén. Kết quả thực nghiệm của ba nhóm bài toán GA sẽ được đối chiếu với một mốc so sánh tương ứng: giá trị cực tiểu toàn cục đã biết của các hàm benchmark cho bài toán không ràng buộc, nghiệm tối ưu công bố của bộ benchmark CEC 2006 cho bài toán có ràng buộc, và nghiệm tối ưu công bố của TSPLIB95 cho bài toán người du lịch. Riêng kết quả của GP cho bài toán hồi quy symbolic sẽ được đối chiếu với phương pháp hồi quy tuyến tính, đóng vai trò mốc so sánh cơ sở (baseline).

Chương 2

Thực nghiệm

Chương này trình bày việc áp dụng các nguyên lý đã trình bày ở Chương 1 vào bốn nhóm bài toán cụ thể. Mỗi nhóm sử dụng một cách mã hóa khác nhau: tối ưu không ràng buộc dùng mã hóa số thực (Mục 2.1), tối ưu có ràng buộc cũng dùng mã hóa số thực nhưng kết hợp thêm cơ chế xử lý ràng buộc (Mục 2.2), bài toán người du lịch dùng mã hóa hoán vị (Mục 2.3), và hồi quy symbolic dùng mã hóa dạng cây biểu thức theo nguyên lý Lập trình Di truyền (Mục 2.4). Với mỗi bài toán, phát biểu toán học đầy đủ được trình bày trước, sau đó là phương pháp so sánh được sử dụng và bảng kết quả thực nghiệm.

2.1 Tối ưu không ràng buộc

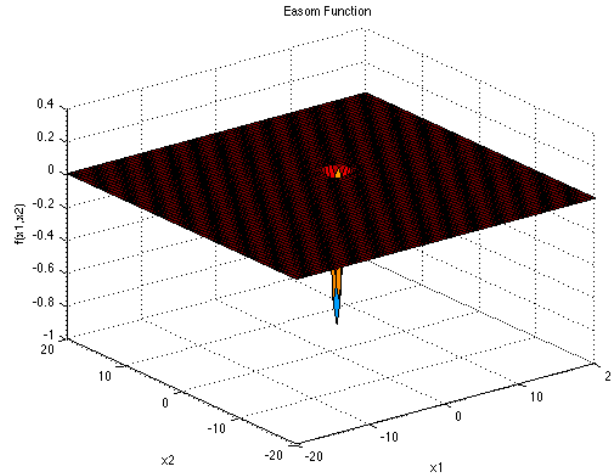
Năm hàm benchmark kinh điển của tối ưu không ràng buộc được sử dụng để đánh giá khả năng tìm cực trị toàn cục của GA. Cả năm hàm đều có hai biến x, y và một cực tiểu toàn cục đã biết trước, cho phép so sánh trực tiếp giá trị GA tìm được với giá trị đúng. Với mỗi hàm, đồ thị ba chiều và đường hội tụ của GA được đặt trên dưới nhau trong cùng một hình. Miền tìm kiếm của từng hàm được lấy theo bộ giá trị chuẩn công bố tại <https://www.sfu.ca/~ssurjano/optimization.html> [8], nơi người đọc có thể tham khảo thêm công thức và đặc trưng chi tiết của từng hàm benchmark.

Hàm Easom có công thức

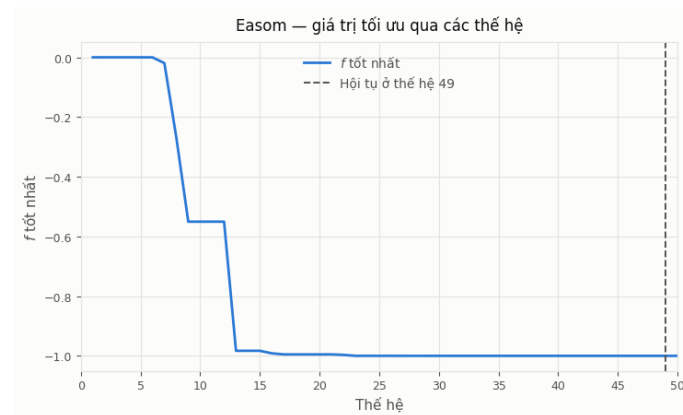
$$f(x, y) = -\cos(x) \cos(y) \exp(-(x - \pi)^2 - (y - \pi)^2), \quad (2.1)$$

với miền tìm kiếm $x, y \in [-100, 100]$ và cực tiểu toàn cục $f(\pi, \pi) = -1$. Khó khăn của hàm này nằm ở chỗ thừa số $\exp(-(x - \pi)^2 - (y - \pi)^2)$ giảm rất nhanh về 0 khi rời khỏi điểm (π, π) : chỉ cần cách tâm khoảng ba đơn vị, giá trị hàm đã nhỏ hơn 10^{-4} và coi như bằng 0. Vùng thực sự mang thông tin dẫn đường vì thế chiếm chưa tới 0,1% diện tích miền tìm kiếm $[-100, 100]^2$, toàn bộ phần còn lại gần như là một mặt phẳng

tuyệt đối. Ở vùng phẳng đó, đạo hàm theo mọi hướng đều xấp xỉ 0, nên các phương pháp dựa trên gradient không có cơ sở nào để xác định nên di chuyển về đâu — đây là tình huống bất lợi nhất đối với nhóm phương pháp này. Hàm Easom vì vậy kiểm tra đúng một khả năng: rải điểm đủ rộng để tìm ra vùng chứa nghiệm, chứ không phải khả năng tinh chỉnh sau khi đã vào gần nghiệm.



(a) Hàm Easom trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.1: Hàm Easom: đồ thị trong không gian ba chiều và diễn biến giá trị tốt nhất qua các thế hệ. Đường nét đứt đánh dấu thế hệ mà GA chốt được nghiệm tốt nhất.

Hàm Rastrigin có công thức

$$f(x, y) = 20 + x^2 + y^2 - 10 \cos(2\pi x) - 10 \cos(2\pi y), \quad (2.2)$$

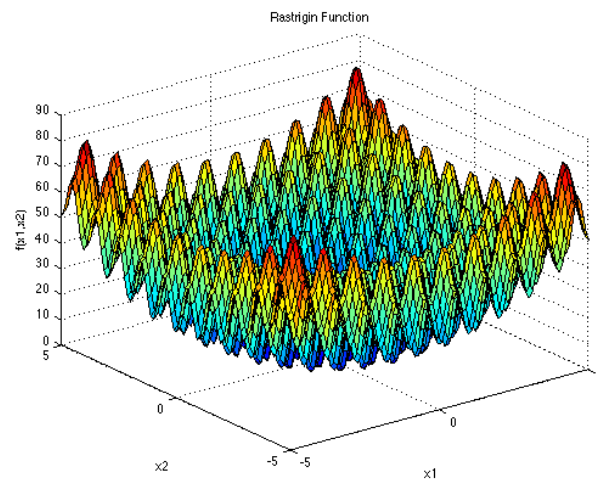
với miền tìm kiếm $x, y \in [-5.12, 5.12]$ và cực tiểu toàn cục $f(0, 0) = 0$.

Bề mặt của hàm không trơn theo nghĩa trực quan mà gồm rất nhiều gợn sóng liên tiếp, tạo thành các cực tiểu và cực đại cục bộ xen kẽ nhau. Do hai số hạng cosin có chu kỳ bằng 1, các cực tiểu cục bộ này xuất hiện gần các điểm có tọa độ nguyên; trong

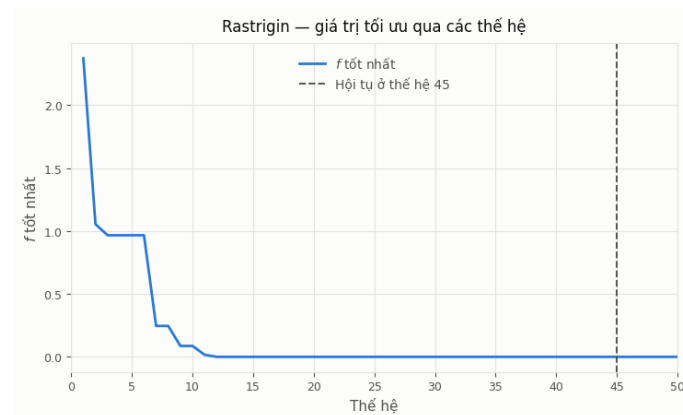
miền $[-5.12, 5.12]^2$ có khoảng 121 cực tiểu cục bộ như vậy, bao quanh dày đặc cực tiểu toàn cục tại gốc tọa độ.

Cấu trúc này khiến Rastrigin trở thành bài toán *đa cực trị* điển hình, khó đối với các phương pháp chỉ tìm kiếm trong phạm vi lân cận: xuất phát từ một điểm ngẫu nhiên, phương pháp cực bộ gần như chắc chắn hội tụ về cực tiểu cục bộ gần nhất rồi dừng lại tại đó, dù vẫn còn một nghiệm tốt hơn ở nơi khác trong không gian tìm kiếm. Nói cách khác, thuật toán dễ bị mắc kẹt tại một cực tiểu cục bộ thay vì tìm được cực tiểu toàn cục.

Đặc điểm này cũng chính là lý do Rastrigin phù hợp để minh họa ưu điểm của GA. Thay vì chỉ cải thiện một nghiệm duy nhất, GA duy trì đồng thời nhiều cá thể trong quần thể và liên tục sinh ra nghiệm mới qua lai ghép và đột biến; đột biến với biên độ phù hợp có thể đưa cá thể thoát khỏi vùng đang bị mắc kẹt để khám phá các vùng khác của không gian tìm kiếm. Nhờ đó, GA có khả năng vượt qua các cực tiểu cục bộ và tăng cơ hội tìm được cực tiểu toàn cục tại gốc tọa độ.



(a) Hàm Rastrigin trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.2: Hàm Rastrigin: mạng lưới cực tiểu cục bộ dày đặc quanh cực tiểu toàn cục, và diễn biến giá trị tốt nhất qua các thế hệ.

Hàm Rosenbrock có công thức

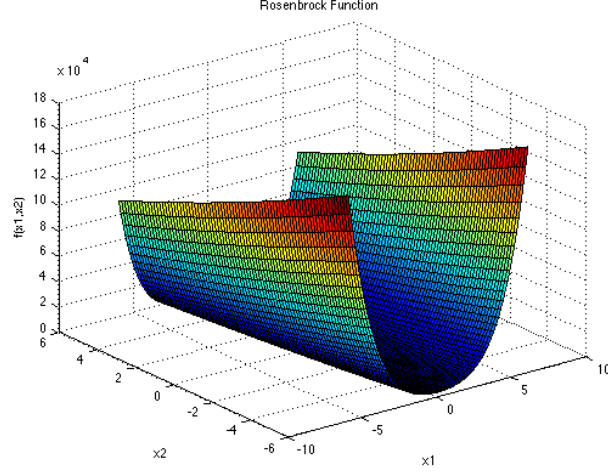
$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad (2.3)$$

với miền tìm kiếm $x, y \in [-5, 10]$ và cực tiểu toàn cục $f(1, 1) = 0$.

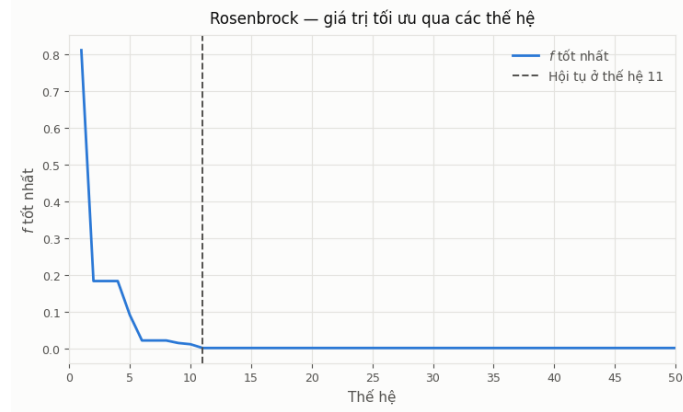
Khác với bốn hàm còn lại, Rosenbrock không có nhiều cực tiểu cục bộ trong miền xét; khó khăn ở đây không nằm ở việc thuật toán bị mắc kẹt tại một cực tiểu cục bộ, mà nằm ở hình dạng của vùng dẫn đến nghiệm tối ưu. Số hạng $100(y - x^2)^2$ phạt rất nặng mọi điểm (x, y) lệch khỏi đường cong $y = x^2$, khiến vùng có giá trị hàm thấp co lại thành một thung lũng dài, hẹp và cong, với nghiệm tối ưu $(1, 1)$ nằm ở đáy.

Dọc theo hai bên thung lũng, giá trị hàm tăng rất nhanh khi lệch khỏi đường cong $y = x^2$, trong khi dọc theo đáy thung lũng giá trị hàm lại thay đổi rất chậm. Vì vậy, thuật toán phải thay đổi x và y một cách *phối hợp* theo đúng đường cong này để tiến dần về $(1, 1)$, thay vì chỉ cải thiện từng biến một cách độc lập.

Do đó, Rosenbrock kiểm tra một khả năng khác của GA so với Rastrigin: nếu Rastrigin kiểm tra khả năng thoát khỏi một mạng lưới dày đặc các cực tiểu cục bộ, thì Rosenbrock kiểm tra khả năng *dò theo* một không gian hẹp, cong và khó tiếp cận sau khi đã tìm ra đúng vùng chứa nghiệm. Đây là ví dụ điển hình cho trường hợp bài toán không có nhiều bẫy cục bộ nhưng vẫn khó tối ưu do cấu trúc của không gian tìm kiếm.



(a) Hàm Rosenbrock trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.3: Hàm Rosenbrock: thung lũng hẹp và cong, và diễn biến giá trị tốt nhất qua các thế hệ — GA chốt kỷ lục ngay ở thế hệ 11 rồi không cải thiện thêm trong 39 thế hệ còn lại.

Hàm Ackley có công thức

$$f(x, y) = -20 \exp\left(-0.2\sqrt{0.5(x^2 + y^2)}\right) - \exp(0.5 \cos(2\pi x) + 0.5 \cos(2\pi y)) + 20 + e, \quad (2.4)$$

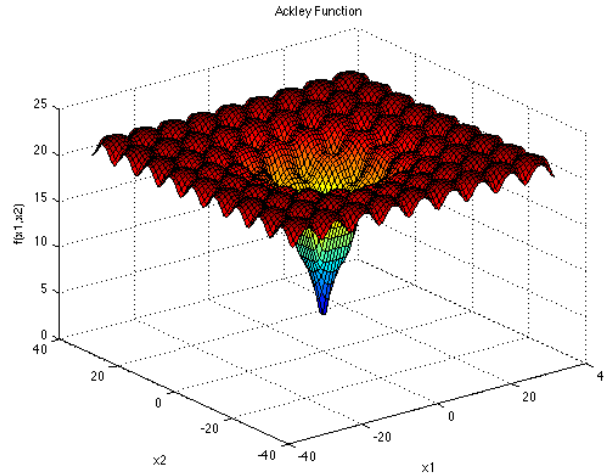
với miền tìm kiếm $x, y \in [-32.768, 32.768]$ và cực tiểu toàn cục $f(0, 0) = 0$.

Bề mặt của hàm có thể hình dung như một phễu lớn phủ nhiều gợn sóng nhỏ. Số hạng mũ thứ nhất tạo ra xu thế tổng thể hướng về gốc tọa độ: càng xa $(0, 0)$, giá trị hàm nhìn chung càng lớn — đây là thông tin có ích, vì nó chỉ đúng hướng cần tiến về nghiệm. Số hạng cosin phủ thêm lên bề mặt đó vô số gợn sóng và cực tiểu cục bộ nhỏ, khiến quá trình tìm kiếm có thể bị nhiễu hoặc dừng lại tại những vị trí chưa tối ưu.

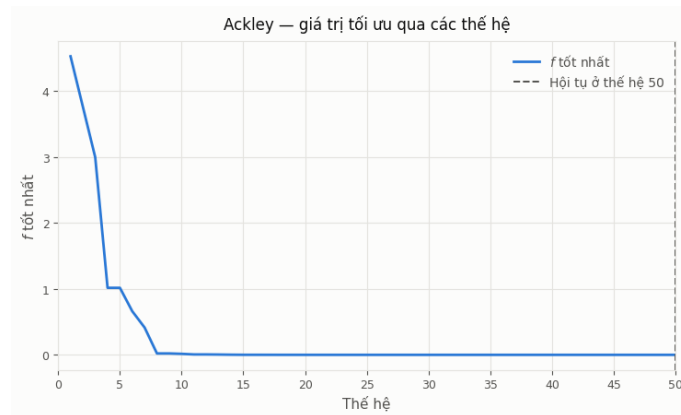
Do đó, khó khăn của Ackley nằm ở việc phân biệt xu thế toàn cục giữa nhiều biến động cục bộ: thuật toán vừa phải tận dụng xu thế chung hướng về gốc tọa độ, vừa

phải duy trì khả năng khám phá để không bị mắc kẹt bởi các gợn sóng nhỏ. Với GA, lai ghép và đột biến liên tục tạo ra cá thể mới ở những vị trí khác nhau, qua đó tăng khả năng vượt qua các cực tiểu cục bộ và tiếp tục tiến về vùng tối ưu.

Như vậy, nếu Rastrigin kiểm tra khả năng thoát khỏi một mạng lưới dày đặc các cực tiểu cục bộ, còn Rosenbrock kiểm tra khả năng dò theo một thung lũng hẹp và cong, thì Ackley kiểm tra khả năng nhận diện xu thế tối ưu ở quy mô lớn trong khi không bị các biến động nhỏ ở quy mô cục bộ chi phối.



(a) Hàm Ackley trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.4: Hàm Ackley: phễu lớn ở thang toàn cục phủ gợn sóng cực tiểu cục bộ ở thang nhỏ, và diễn biến giá trị tốt nhất qua các thế hệ.

Hàm Schwefel có công thức

$$f(x, y) = 837.9658 - x \sin \sqrt{|x|} - y \sin \sqrt{|y|}, \quad (2.5)$$

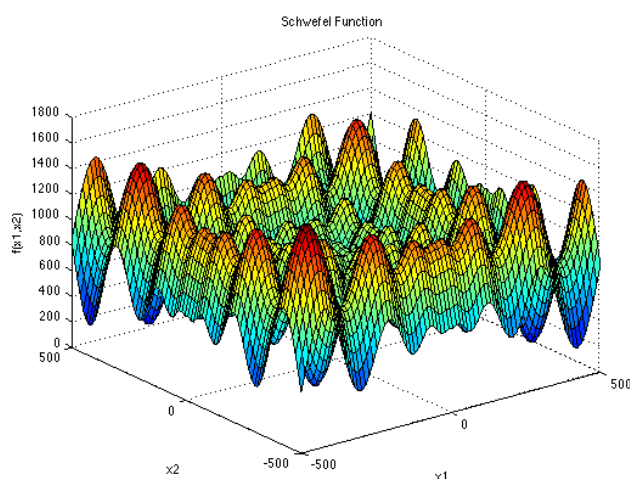
với miền tìm kiếm $x, y \in [-500, 500]$ và cực tiểu toàn cục $f(420.9687, 420.9687) \approx 0$.

Đặc điểm của Schwefel là vùng có vẻ tốt nhất tại một vị trí có thể không dẫn đến nghiệm tối ưu thực sự: các cực tiểu cục bộ của nó phân bố không đều trong toàn miền

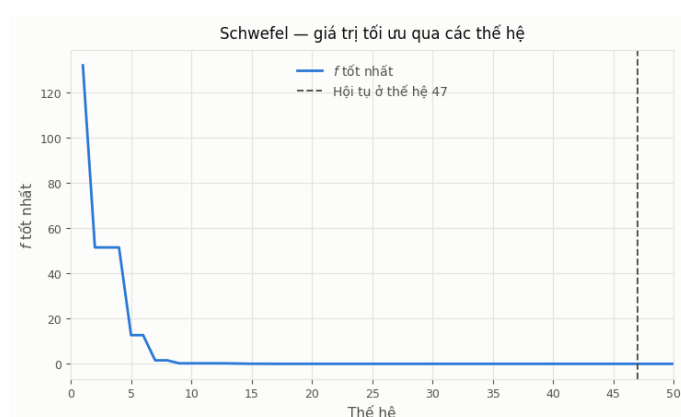
tìm kiếm, và cực tiểu cục bộ tốt thứ nhì lại nằm rất xa cực tiểu toàn cục. Do đó, nếu thuật toán chỉ tập trung khai thác một vùng đang cho kết quả tốt, nó có thể nhanh chóng đạt được một nghiệm khá tốt nhưng bỏ qua nghiệm tốt nhất nằm ở một khu vực hoàn toàn khác.

Thêm vào đó, cực tiểu toàn cục $(420,9687, 420,9687)$ nằm khá gần biên của miền tìm kiếm, nên thuật toán không nên mặc định rằng nghiệm tốt nằm gần tâm không gian tìm kiếm; nếu cơ chế xử lý biên vô tình co cá thể về phía tâm miền, khả năng tìm thấy nghiệm tối ưu sẽ giảm.

Vì vậy, Schwefel chủ yếu kiểm tra khả năng duy trì sự đa dạng và thăm dò trên phạm vi rộng của không gian tìm kiếm, thay vì hội tụ sớm vào một vùng có vẻ hứa hẹn. Đây là một điểm mạnh của GA, vì quần thể gồm nhiều cá thể cho phép thuật toán tiếp tục khám phá nhiều khu vực khác nhau thay vì chỉ đi theo một hướng duy nhất.



(a) Hàm Schwefel trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.5: Hàm Schwefel: cực tiểu toàn cục nằm gần biên miền tìm kiếm, xa vùng trông có vẻ tốt quanh gốc tọa độ, và diễn biến giá trị tốt nhất qua các thế hệ.

Đối với mỗi hàm benchmark, GA được thực nghiệm trên cùng một thiết lập với quần thể 500 cá thể và tối đa 50 thế hệ. Các cá thể được giữ trong miền tìm kiếm của từng hàm và thuật toán luôn chạy đủ 50 thế hệ. Cột *số thế hệ thỏa mãn* biểu thị thế hệ sớm nhất đạt được giá trị tốt nhất cuối cùng f^* , không phải thời điểm thuật toán dừng.

Bảng 2.1: Kết quả GA trên năm hàm benchmark không ràng buộc

Hàm	Thời gian (s)	Thế hệ	$f_{\text{thực}}$	f^*	x^*	y^*
Easom	0,71	48	-1	-1,0000000000	3,141593	3,141593
Rastrigin	0,77	45	0	0,0000000000	$8,00 \times 10^{-10}$	$-2,11 \times 10^{-9}$
Rosenbrock	0,70	11	0	0,0009051359	1,008839	1,014881
Ackley	0,75	50	0	$3,344 \times 10^{-10}$	$-7,55 \times 10^{-11}$	$-9,10 \times 10^{-11}$
Schwefel	0,73	47	≈ 0	$2,5455 \times 10^{-5}$	420,968746	420,968746

Nhận xét và đánh giá kết quả

Kết quả cho thấy GA tìm được nghiệm toàn cục hoặc nghiệm rất gần tối ưu trên cả năm hàm. Easom và Rastrigin đạt đúng giá trị tối ưu; Ackley và Schwefel cũng đạt sai số rất nhỏ. Đáng chú ý, GA vẫn tìm được nghiệm tốt trên Easom và Schwefel, cho thấy khả năng khám phá rộng của tìm kiếm dựa trên quần thể.

Rosenbrock là trường hợp khó nhất khi chỉ đạt $f^* = 9,05 \times 10^{-4}$. GA nhanh chóng tiến vào vùng gần nghiệm tối ưu nhưng sau đó hầu như không cải thiện thêm. Nguyên nhân là thung lũng của Rosenbrock hẹp và cong, đòi hỏi quá trình tinh chỉnh chính xác dọc theo đáy thung lũng, trong khi các phép biến đổi của GA không thực sự hiệu quả ở giai đoạn này.

Nhìn chung, kết quả cho thấy GA có ưu thế rõ rệt trong khám phá và định vị vùng nghiệm, nhưng khả năng tinh chỉnh nghiệm trong những vùng hẹp và có cấu trúc đặc biệt còn hạn chế. Đây cũng là cơ sở để kết hợp GA với tìm kiếm cục bộ trong các bài toán cần độ chính xác cao ở giai đoạn cuối.

2.2 Tối ưu có ràng buộc

Với bài toán có ràng buộc, không phải mọi điểm trong miền tìm kiếm đều hợp lệ; do đó, thuật toán phải xét tính khả thi trước khi tối ưu hàm mục tiêu, vì nghiệm vi phạm ràng buộc dù có giá trị mục tiêu tốt vẫn không có ý nghĩa.

Thực nghiệm sử dụng bộ benchmark CEC 2006 của Liang và cộng sự (https://cw.fel.cvut.cz/b241/_media/courses/a0m33eoa/cviceni/2006_problem_definitions_and_evaluation_criteria_for_the_cec_2006_special_session_on_constraint_real-parameter_optimization.pdf [9]), gồm 24 bài toán tối ưu có ràng buộc kèm nghiệm tối ưu. Năm bài toán g01, g06, g08, g11 và g15 được chọn nhằm bao quát nhiều dạng ràng buộc, mức độ chặt hẹp của miền khả thi và vị trí của nghiệm tối

ưu so với biên.

Theo bộ benchmark CEC 2006, ràng buộc bất đẳng thức yêu cầu $g_i(x) \leq 0$, trong khi ràng buộc đẳng thức sử dụng dung sai $|h_j(x)| - \varepsilon \leq 0$ với $\varepsilon = 10^{-4}$. Với ràng buộc đẳng thức, việc yêu cầu $h_j(x) = 0$ một cách tuyệt đối là không thực tế đối với thuật toán số, vì chỉ cần sai lệch rất nhỏ cũng khiến nghiệm bị xem là không hợp lệ. Chẳng hạn, nghiệm công bố của g11 và g15 cũng nằm ngay tại mức sai số này.

GA mã hóa số thực được kết hợp quy tắc khả thi của Deb và ngưỡng ε giảm dần (Mục 1.3), sử dụng 1000 cá thể trong 500 thế hệ. Sau GA, một bước tìm kiếm cục bộ theo từng trục tọa độ được thực hiện từ nghiệm tốt nhất: chỉ chấp nhận điểm khả thi có giá trị mục tiêu tốt hơn; khi không còn cải thiện, bước dịch chuyển được giảm một nửa và tiếp tục tìm kiếm. Mô hình kết hợp này cũng được áp dụng cho bài toán người du lịch ở Mục 2.3. Trong đó, GA đảm nhiệm khám phá toàn cục, còn tìm kiếm cục bộ đảm nhiệm tinh chỉnh nghiệm, đặc biệt khi nghiệm nằm sát biên miền khả thi. Kết quả của cả hai giai đoạn được báo cáo riêng để đánh giá đóng góp của từng bước. Kết quả chi tiết trên từng bài toán được trình bày lần lượt dưới đây.

Bài toán g01 có hàm mục tiêu bậc hai trên mười ba biến, chịu chín ràng buộc bất đẳng thức đều tuyến tính:

$$\begin{aligned} \min_{x \in \mathbb{R}^{13}} \quad & f(x) = 5 \sum_{i=1}^4 x_i - 5 \sum_{i=1}^4 x_i^2 - \sum_{i=5}^{13} x_i \\ \text{với điều kiện} \quad & 2x_1 + 2x_2 + x_{10} + x_{11} - 10 \leq 0 \\ & 2x_1 + 2x_3 + x_{10} + x_{12} - 10 \leq 0 \\ & 2x_2 + 2x_3 + x_{11} + x_{12} - 10 \leq 0 \\ & -8x_1 + x_{10} \leq 0 \\ & -8x_2 + x_{11} \leq 0 \\ & -8x_3 + x_{12} \leq 0 \\ & -2x_4 - x_5 + x_{10} \leq 0 \\ & -2x_6 - x_7 + x_{11} \leq 0 \\ & -2x_8 - x_9 + x_{12} \leq 0 \end{aligned} \tag{2.6}$$

với miền biến thiên $0 \leq x_i \leq 1$ cho $i = 1, \dots, 9$, $0 \leq x_i \leq 100$ cho $i = 10, 11, 12$ và $0 \leq x_{13} \leq 1$. Nghiệm tối ưu công bố là

$$x^* = (1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1), \quad f(x^*) = -15,$$

tại đó sáu trong chín ràng buộc đạt đẳng thức.

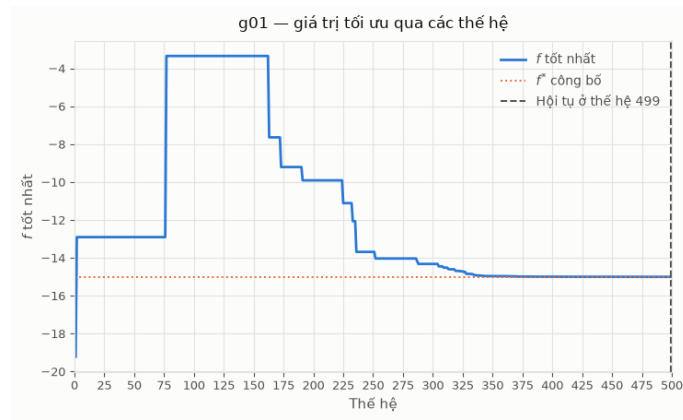
Đây là bài toán có số biến lớn nhất trong năm bài. Điểm thuận lợi nằm ở chỗ mọi ràng buộc đều tuyến tính, nên miền khả thi là một đa diện lồi và vùng lân cận nghiệm

tối ưu vẫn còn thể tích đáng kể. Nhờ vậy quần thể di chuyển tương đối dễ dàng ngay cả khi đã tiến sát nghiệm.

Bảng 2.2: Kết quả trên bài toán g01

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm	Số thể hệ thỏa mãn
-15,000000	-14,999204	-15,000000	$1,77 \times 10^{-12}$	0	499

GA đơn thuần đạt sai lệch tương đối $5,3 \times 10^{-5}$, một kết quả tốt đối với không gian mười ba chiều. Bước tinh chỉnh đưa sai lệch xuống $1,77 \times 10^{-12}$, tức trùng với nghiệm công bố trong giới hạn biểu diễn số thực dấu phẩy động.



Hình 2.6: Đường hội tụ của GA trên bài toán g01

Bài toán g06 chỉ có hai biến nhưng hàm mục tiêu bậc ba và cả hai ràng buộc đều phi tuyến:

$$\begin{aligned}
 \min_{x \in \mathbb{R}^2} \quad & f(x) = (x_1 - 10)^3 + (x_2 - 20)^3 \\
 \text{với điều kiện} \quad & -(x_1 - 5)^2 - (x_2 - 5)^2 + 100 \leq 0 \\
 & (x_1 - 6)^2 + (x_2 - 5)^2 - 82,81 \leq 0
 \end{aligned} \tag{2.7}$$

với $13 \leq x_1 \leq 100$ và $0 \leq x_2 \leq 100$. Nghiệm tối ưu công bố là

$$x^* = (14,095000; 0,842961), \quad f(x^*) = -6961,813876,$$

tại đó cả hai ràng buộc đều đạt đẳng thức.

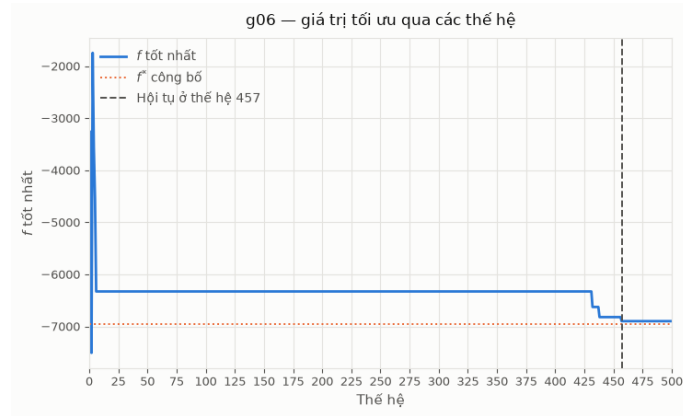
Hai ràng buộc mô tả phần bên ngoài một đường tròn và phần bên trong một đường tròn khác, nên miền khả thi là một dải hình lưỡi liềm rất hẹp, chỉ chiếm 0,0066% thể tích miền tìm kiếm. Nghiệm tối ưu nằm đúng tại giao điểm của hai đường tròn, tức một điểm không chiều. Bên cạnh đó, chuẩn của gradient hàm mục tiêu tại nghiệm xấp xỉ 1102, lớn hơn hai bậc độ lớn so với các bài toán còn lại. Hệ quả là một sai lệch nhỏ

về vị trí bị khuếch đại thành sai lệch lớn về giá trị hàm mục tiêu. Đây là bài toán duy nhất trong năm bài hội tụ đồng thời cả hai yếu tố bất lợi: miền khả thi cực hẹp quanh nghiệm và hàm mục tiêu rất dốc.

Bảng 2.3: Kết quả trên bài toán g06

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm	Số thể hệ thỏa mãn
-6961,813876	-6898,462524	-6961,813876	$2,64 \times 10^{-8}$	0	457

GA đơn thuần dừng lại ở $-6898,462524$, cách nghiệm công bố 63,35 đơn vị, tương ứng sai lệch tương đối $9,1 \times 10^{-3}$. Cần lưu ý rằng con số này phản ánh độ dốc của hàm mục tiêu nhiều hơn là chất lượng định vị của GA: quy đổi qua chuẩn gradient, sai lệch đó tương ứng với sai số vị trí chỉ khoảng 0,06% bề rộng miền tìm kiếm. Bước tinh chỉnh cục bộ đưa kết quả về $2,64 \times 10^{-8}$, cải thiện hơn chín bậc độ lớn. Đây là bài toán mà việc kết hợp tìm kiếm cục bộ phát huy tác dụng rõ rệt nhất.



Hình 2.7: Đường hội tụ của GA trên bài toán g06

Bài toán g08 có hàm mục tiêu chứa hàm lượng giác và phân thức:

$$\min_{x \in \mathbb{R}^2} f(x) = -\frac{\sin^3(2\pi x_1) \sin(2\pi x_2)}{x_1^3(x_1 + x_2)} \quad (2.8)$$

với điều kiện $x_1^2 - x_2 + 1 \leq 0$

$$1 - x_1 + (x_2 - 4)^2 \leq 0$$

với $0 \leq x_1 \leq 10$ và $0 \leq x_2 \leq 10$. Nghiệm tối ưu công bố là

$$x^* = (1,227971; 4,245373), \quad f(x^*) = -0,095825.$$

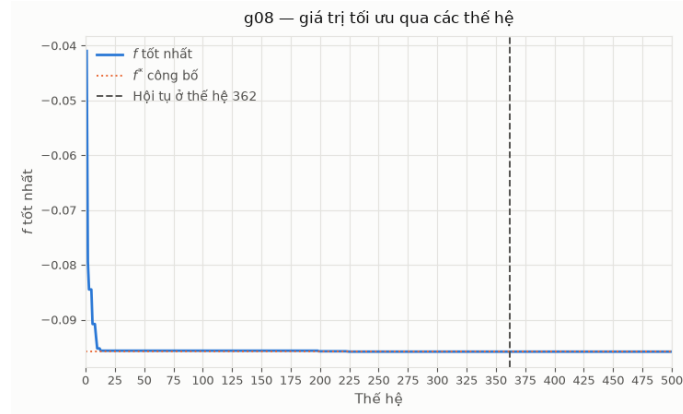
Điểm khác biệt căn bản của bài toán này là nghiệm tối ưu nằm hẳn bên trong miền khả thi, không ràng buộc nào đạt đẳng thức. Do đó nghiệm là một điểm dừng của

hàm mục tiêu và gradient tại đó bằng không, khiến sai lệch về vị trí hầu như không chuyển thành sai lệch về giá trị hàm. Miền khả thi cũng rộng rãi hơn hẳn bốn bài còn lại, chiếm 0,8560% miền tìm kiếm.

Bảng 2.4: Kết quả trên bài toán g08

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm	Số thể hệ thỏa mãn
-0,095825	-0,095825	-0,095825	$2,78 \times 10^{-17}$	0	362

GA đạt nghiệm công bố ngay ở giai đoạn đầu với sai lệch $2,78 \times 10^{-17}$, tức đã chạm ngưỡng chính xác của số thực dấu phẩy động, và bước tinh chỉnh không còn gì để cải thiện. Kết quả này minh họa rằng độ khó của một bài toán có ràng buộc không nằm ở tính phi tuyến của hàm mục tiêu mà nằm ở vị trí của nghiệm so với biên miền khả thi.



Hình 2.8: Đường hội tụ của GA trên bài toán g08

Bài toán g11 có dạng đơn giản nhất về hình thức nhưng chứa một ràng buộc đẳng thức phi tuyến:

$$\min_{x \in \mathbb{R}^2} f(x) = x_1^2 + (x_2 - 1)^2 \quad (2.9)$$

với điều kiện $x_2 - x_1^2 = 0$

với $-1 \leq x_1 \leq 1$ và $-1 \leq x_2 \leq 1$. Nghiệm tối ưu công bố là

$$x^* = (-0,707036; 0,500000), \quad f(x^*) = 0,7499.$$

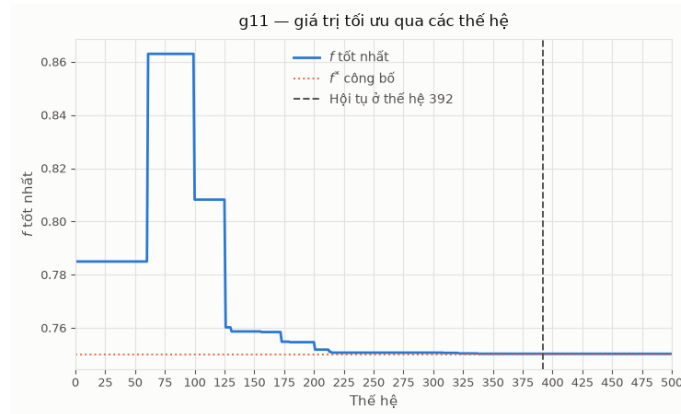
Miền khả thi ở đây là một đường cong parabol trong mặt phẳng, tức một tập có diện tích bằng không. Không một điểm sinh ngẫu nhiên nào rơi trúng miền này, nên toàn bộ việc tiếp cận miền khả thi phụ thuộc vào cơ chế nói lỏng ngưỡng ε của thuật toán.

Bài toán này cho phép kiểm chứng nghiệm bằng giải tích. Thế $x_2 = x_1^2$ vào hàm mục tiêu và đặt $u = x_1^2 \geq 0$, ta được $u + (u - 1)^2 = u^2 - u + 1$, đạt cực tiểu tại $u = 1/2$ với giá trị đúng bằng 0,75. Như vậy cực tiểu chính xác của bài toán là 0,75, trong khi giá trị công bố 0,7499 lại nhỏ hơn con số đó. Nguyên nhân là nghiệm công bố không nằm trên đường cong mà lệch khỏi nó đúng bằng dung sai $\varepsilon = 10^{-4}$, và độ lệch ấy được tận dụng theo hướng làm giảm hàm mục tiêu.

Bảng 2.5: Kết quả trên bài toán g11

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm	Số thế hệ thỏa mãn
0,749900	0,750268	0,750259	$3,59 \times 10^{-4}$	0	392

GA hội tụ về 0,750259, tức cách cực tiểu giải tích 0,75 một khoảng $2,6 \times 10^{-4}$ và cách giá trị công bố $3,59 \times 10^{-4}$. Phần chênh lệch còn lại mang tính cấu trúc chứ không phải do thuật toán hội tụ kém: muốn đạt được đúng con số 0,7499 thì nghiệm phải nằm ở rìa ngoài của dải dung sai, điều mà một thuật toán tìm nghiệm khả thi không hướng tới. Bước tinh chỉnh cải thiện không đáng kể vì miền khả thi là một dải cong rất hẹp, trong đó các bước dịch chuyển song song với trục tọa độ nhanh chóng đi ra ngoài miền và bị loại.



Hình 2.9: Đường hội tụ của GA trên bài toán g11

Bài toán g15 có ba biến cùng hai ràng buộc đẳng thức, một phi tuyến và một tuyến tính:

$$\begin{aligned}
 \min_{x \in \mathbb{R}^3} \quad & f(x) = 1000 - x_1^2 - 2x_2^2 - x_3^2 - x_1x_2 - x_1x_3 \\
 \text{với điều kiện} \quad & x_1^2 + x_2^2 + x_3^2 - 25 = 0 \\
 & 8x_1 + 14x_2 + 7x_3 - 56 = 0
 \end{aligned} \tag{2.10}$$

với $0 \leq x_i \leq 10$ cho $i = 1, 2, 3$. Nghiệm tối ưu công bố là

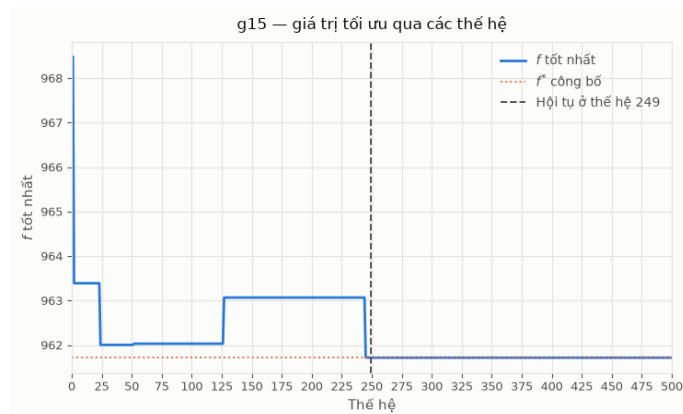
$$x^* = (3,512128; 0,216988; 3,552179), \quad f(x^*) = 961,715022.$$

Hai ràng buộc đẳng thức mô tả một mặt cầu và một mặt phẳng, giao của chúng là một đường tròn trong không gian ba chiều. Xét về thể tích, đây là miền khả thi chật hẹp nhất trong năm bài toán. Bù lại, hàm mục tiêu khá thoải: chuẩn gradient tại nghiệm chỉ khoảng 15,8, nên sai lệch về vị trí không bị khuếch đại mạnh như ở g06.

Bảng 2.6: Kết quả trên bài toán g15

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm	Số thế hệ thỏa mãn
961,715022	961,715374	961,715308	$2,86 \times 10^{-4}$	0	249

Sai lệch tương đối chỉ $3,0 \times 10^{-7}$, thuộc nhóm kết quả tốt nhất trong năm bài toán mặc dù miền khả thi hẹp nhất. Điều đáng chú ý là kết quả này đạt được ngay từ giai đoạn GA; cũng như ở g11, bước tinh chỉnh cục bộ không phát huy nhiều tác dụng khi miền khả thi là một đường cong hẹp.



Hình 2.10: Đường hội tụ của GA trên bài toán g15

Nhận xét và đánh giá kết quả

Bảng 2.7 tổng hợp kết quả trên cả năm bài toán. Mọi nghiệm báo cáo đều khả thi tuyệt đối theo định nghĩa của bộ benchmark, với mức vi phạm bằng 0.

Kết quả cho thấy độ khó của một bài toán tối ưu có ràng buộc đối với GA không được quyết định bởi số biến hay bậc của hàm mục tiêu, mà bởi hai yếu tố thuộc về hình học của bài toán tại lân cận nghiệm tối ưu.

Yếu tố thứ nhất là thể tích miền khả thi quanh nghiệm. Khi nghiệm nằm sâu bên trong miền khả thi như ở g08, quần thể di chuyển tự do và GA đạt độ chính xác giới hạn của số thực ngay từ giai đoạn tiến hóa. Khi nghiệm nằm tại một đỉnh của đa diện

Bảng 2.7: Tổng hợp kết quả GA trên năm bài toán chuẩn CEC 2006

Bài toán	f^* công bố	f^* (GA)	f^* (GA) + tinh chỉnh	Sai lệch tương đối	Thời gian (s)
g01	-15,000000	-14,999204	-15,000000	$1,2 \times 10^{-13}$	18,7
g06	-6961,813876	-6898,462524	-6961,813876	$3,8 \times 10^{-12}$	14,1
g08	-0,095825	-0,095825	-0,095825	$2,9 \times 10^{-16}$	13,2
g11	0,749900	0,750268	0,750259	$4,8 \times 10^{-4}$	13,3
g15	961,715022	961,715374	961,715308	$3,0 \times 10^{-7}$	13,8

lỗi như ở g01, vùng lân cận vẫn còn thể tích đáng kể nên GA tiếp cận tốt. Ngược lại, khi nghiệm nằm tại giao điểm của hai mặt cong như ở g06, hoặc trên một đường cong có thể tích bằng không như ở g11 và g15, các phép lai ghép và đột biến ngẫu nhiên hầu như luôn sinh ra cá thể bất khả thi và bị loại bỏ, khiến quá trình cải thiện chững lại.

Yếu tố thứ hai là độ dốc của hàm mục tiêu tại nghiệm, đại lượng quyết định một sai lệch về vị trí được khuếch đại bao nhiêu lần thành sai lệch về giá trị hàm. Chuẩn gradient tại nghiệm của g06 lớn gấp khoảng bảy mươi lần so với g15 và gấp một trăm lần so với g01, giải thích vì sao cùng một mức chính xác về vị trí lại cho ra sai lệch chênh nhau nhiều bậc độ lớn giữa các bài toán.

Hai yếu tố này độc lập với nhau, và g06 là bài toán duy nhất mà cả hai cùng bất lợi. Đó cũng là bài toán mà giai đoạn tinh chỉnh cục bộ mang lại cải thiện lớn nhất, đưa sai lệch từ 63,35 xuống $2,64 \times 10^{-8}$. Nhìn rộng hơn, sự phân công giữa hai giai đoạn đã được xác nhận qua thực nghiệm: GA giải quyết phần việc định vị vùng chứa nghiệm trên một không gian mà phần lớn là bất khả thi, còn tìm kiếm cục bộ giải quyết phần việc tiếp cận chính xác nghiệm nằm sát biên. Không giai đoạn nào thay thế được giai đoạn kia, và chi phí tính toán của giai đoạn tinh chỉnh không đáng kể so với giai đoạn tiến hóa.

Riêng với g11 và g15, phần sai lệch còn lại chủ yếu bắt nguồn từ dung sai $\varepsilon = 10^{-4}$ dành cho ràng buộc đẳng thức chứ không phải từ năng lực hội tụ của thuật toán. Như đã phân tích ở phần bài toán g11 phía trên, giá trị công bố của g11 còn nhỏ hơn cực tiểu chính xác của bài toán, nên khoảng cách giữa kết quả thu được và giá trị công bố không phản ánh sai số của phương pháp.

2.3 Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ

Bài toán người du lịch (Traveling Salesman Problem — TSP) được phát biểu như sau: cho n địa điểm và ma trận khoảng cách $D = (d_{ij})_{n \times n}$ giữa mọi cặp địa điểm, tìm một hoán vị $\pi = (\pi_1, \dots, \pi_n)$ của n địa điểm sao cho tổng độ dài lộ trình khép kín đi qua

tất cả các địa điểm đúng một lần là nhỏ nhất:

$$\min_{\pi} L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1}. \quad (2.11)$$

Bài toán này được mã hóa dưới dạng hoán vị đã giới thiệu ở Mục 1.2: mỗi cá thể của GA chính là một hoán vị π , dùng lai ghép theo thứ tự và đột biến hoán đổi (Mục 1.3) thay cho các toán tử số thực ở hai mục trước.

Để đánh giá khách quan trên dữ liệu thực thay vì các ví dụ tự tạo quy mô nhỏ, ba bộ dữ liệu chuẩn từ thư viện benchmark TSPLIB95 [7] được sử dụng: berlin52 ($n = 52$ địa điểm tại Berlin), rd100 ($n = 100$ địa điểm ngẫu nhiên), và ch150 ($n = 150$ địa điểm). Mỗi bộ dữ liệu đều có kèm theo lộ trình tối ưu đã được công bố, dùng làm mốc đối chiếu khách quan thay cho phương pháp vét cạn — vốn cần thử $(n - 1)!$ hoán vị và trở nên bất khả thi về mặt tính toán ngay từ n vài chục.

Thử nghiệm ban đầu với GA hoán vị thuần túy (chỉ dùng lai ghép theo thứ tự và đột biến hoán đổi) cho kết quả chênh lệch rất lớn so với nghiệm tối ưu đã công bố, từ 20% đến 84% tùy bộ dữ liệu và tăng dần theo số địa điểm n . Nguyên nhân là hai toán tử này không có cơ chế sửa các đoạn đường bị cắt chéo nhau trong lộ trình — một lộ trình càng có nhiều địa điểm thì càng dễ chứa những đoạn cắt chéo như vậy, và chúng chỉ có thể tạo ra một cấu hình mới ngẫu nhiên chứ không chủ động “gỡ” đoạn cắt chéo đó. Để khắc phục hạn chế này, một chiến lược lai giữa GA và tìm kiếm cục bộ được áp dụng: sau mỗi thế hệ, một tỉ lệ cá thể tốt nhất trong quần thể được tinh chỉnh thêm bằng phép tìm kiếm cục bộ **2-opt** — xét lần lượt từng cặp cạnh trong lộ trình, hoán đổi hai cạnh đó nếu việc hoán đổi làm giảm tổng độ dài — trước khi tiếp tục vòng lặp tiến hóa (chọn lọc, lai ghép, đột biến) như bình thường. Với tỉ lệ tinh chỉnh bằng khoảng 5% quần thể mỗi thế hệ, cả ba bộ dữ liệu đều đạt đúng lộ trình tối ưu đã công bố, như trình bày ở Bảng 2.8.

Bảng 2.8: Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95

Bộ dữ liệu	Số địa điểm (n)	Nghiệm tối ưu	GA	Đạt được ở thế hệ
berlin52	52	7542	7542	8/50
rd100	100	7910	7910	48/50
ch150	150	6528	6528	37/50

Trong quá trình hiệu chỉnh tham số, một quan sát quan trọng được ghi nhận: tỉ lệ cá thể được tinh chỉnh bằng 2-opt cần được đặt theo *phần trăm* của quần thể chứ không phải theo một số lượng cố định. Cụ thể, khi giữ nguyên số lượng cá thể được tinh chỉnh ở một con số tuyệt đối nhưng tăng kích thước quần thể lên, kết quả trở nên tệ hơn so với quần thể nhỏ — nguyên nhân là vì trong một quần thể lớn, số cá thể đã qua tinh chỉnh chiếm tỉ lệ quá nhỏ nên khó lan gen tốt ra phần còn lại của quần thể

thông qua phép chọn lọc. Quan sát này cho thấy các tham số của một chiến lược lai giữa GA và tìm kiếm cục bộ không thể được chọn độc lập với nhau, mà phải cân chỉnh tương thích với quy mô quần thể.

2.4 Hồi quy Symbolic bằng Lập trình Di truyền

Ứng dụng cuối cùng minh họa Lập trình Di truyền (Mục 1.5) cho bài toán hồi quy symbolic trên dữ liệu vật lý thực: các phép đo mật độ hạt, vận tốc, và từ trường của gió mặt trời do vệ tinh WIND của NASA ghi nhận. Ba thành phần từ trường đo được là B_X, B_Y, B_Z , và mật độ hạt đo được là n . Từ bốn đại lượng này, hai đại lượng vật lý phái sinh được chọn làm mục tiêu dự đoán cho GP:

$$|B|^2 = B_X^2 + B_Y^2 + B_Z^2 \quad (\text{bình phương độ lớn từ trường}), \quad (2.12)$$

$$V_A^2 \propto \frac{|B|^2}{n} \quad (\text{vận tốc Alfvén bình phương}). \quad (2.13)$$

Đại lượng thứ nhất là một quan hệ tổng bình phương (dạng Pythagore) giữa ba biến đo được; đại lượng thứ hai kết hợp thêm một phép chia cho biến mật độ hạt.

Điểm mấu chốt khiến hai đại lượng này trở thành phép thử công bằng cho GP là: quan hệ tổng bình phương ở Phương trình (2.12) *không thể* tuyến tính hóa bằng phép biến đổi logarit như các quan hệ dạng lũy thừa thông thường, vì B_X, B_Y, B_Z có thể mang giá trị âm nên không lấy logarit được. Do đó, Hồi quy Tuyến tính áp dụng trực tiếp trên ba đặc trưng thô B_X, B_Y, B_Z gần như chắc chắn không thể biểu diễn được cấu trúc bậc hai này, vì mô hình tuyến tính chỉ có thể biểu diễn một tổ hợp tuyến tính của chính các đặc trưng đó, không tự sinh ra được số hạng bậc hai. Ngược lại, GP có thể tự xây dựng số hạng bậc hai bằng cách nhân một biến với chính nó ngay trong cây biểu thức, mà không cần được cung cấp trước các đặc trưng B_X^2, B_Y^2, B_Z^2 như một phép biến đổi thủ công. Kết quả đối chiếu hai phương pháp trên tập kiểm tra được trình bày ở Bảng 2.9.

Bảng 2.9: Kết quả Hồi quy Tuyến tính và GP trên hai đại lượng vật lý phái sinh

Đại lượng	Phương pháp	MAE	RMSE	R^2
$ B ^2$	Hồi quy Tuyến tính	15.1284	19.9825	0.4784
	GP (hồi quy symbolic)	4.1820	6.0779	0.9517
V_A^2	Hồi quy Tuyến tính	2.7376	3.6804	0.4629
	GP (hồi quy symbolic)	1.0554	1.5486	0.9049

Kết quả cho thấy chênh lệch rõ rệt và nhất quán trên cả hai đại lượng. Hồi quy Tuyến tính chỉ giải thích được khoảng 46% đến 48% phương sai của dữ liệu ($R^2 \approx 0.46\text{--}0.48$),

trong khi GP đạt trên 90% ($R^2 \approx 0.90\text{--}0.95$); đồng thời sai số dự đoán của GP chỉ còn chưa đến một phần ba so với Hồi quy Tuyến tính ở cả hai đại lượng. Đây là kết quả trái ngược hẳn với các bài toán hồi quy dạng lũy thừa từng được thử nghiệm trong quá trình thực hiện tiểu luận — ví dụ định luật Kepler III hay áp suất động của gió mặt trời — là những quan hệ trở thành tuyến tính tuyệt đối sau khi biến đổi logarit, khiến Hồi quy Tuyến tính ở các bài toán đó luôn đạt hoặc vượt GP. Sự tương phản giữa hai nhóm kết quả khẳng định đúng giả thuyết ban đầu: lợi thế thực sự của GP so với hồi quy cổ điển chỉ bộc lộ rõ khi quan hệ cần khám phá có cấu trúc phi tuyến mà phép biến đổi đặc trưng thủ công thông thường, như logarit, không xử lý được.

2.5 Tổng kết Chương

Bốn thực nghiệm trên minh họa một đặc điểm xuyên suốt của Thuật toán Di truyền: tính linh hoạt cao nhờ khả năng thích ứng với nhiều cách mã hóa khác nhau — số thực ở Mục 2.1 và 2.2, hoán vị ở Mục 2.3, và cây biểu thức ở Mục 2.4. Đổi lại tính linh hoạt đó là hiệu quả thấp hơn các phương pháp chuyên dụng khi bài toán có sẵn một cấu trúc mà phương pháp đó khai thác được: ở năm bài toán chuẩn CEC 2006 tại Mục 2.2, GA phải bổ sung một bước tìm kiếm cục bộ mới tiếp cận được những nghiệm nằm sát biên miền khả thi, phần việc mà một phương pháp dựa trên gradient xử lý trực tiếp nhờ khai thác tính khả vi của hàm mục tiêu và ràng buộc. Ngược lại, ở những bài toán mà cấu trúc đó không có sẵn, hoặc phương pháp cổ điển tương ứng không còn đủ mạnh — bài toán người du lịch quy mô lớn cần bổ sung tìm kiếm cục bộ mới đạt được nghiệm tối ưu, hay quan hệ phi tuyến dạng tổng bình phương mà Hồi quy Tuyến tính không thể biểu diễn được — GA và GP thể hiện rõ giá trị thực tiễn của một phương pháp tìm kiếm không đặt giả định trước về cấu trúc bài toán.

Tài liệu tham khảo

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT Press.
4. Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
5. Takahama, T., & Sato, S. (2010). Constrained optimization by the ε constrained differential evolution with gradient-based mutation and feasible elites. *IEEE Congress on Evolutionary Computation (CEC)*.
6. Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.
7. Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376–384.
8. Surjanovic, S., & Bingham, D. (2013). Virtual Library of Simulation Experiments: Test Functions and Datasets. Simon Fraser University. <https://www.sfu.ca/~ssurjano/optimization.html>
9. Liang, J. J., Runarsson, T. P., Mezura-Montes, E., Clerc, M., Suganthan, P. N., Coello Coello, C. A., & Deb, K. (2006). Problem Definitions and Evaluation Criteria for the CEC 2006 Special Session on Constrained Real-Parameter Optimization. https://cw.fel.cvut.cz/b241/_media/courses/a0m33eoa/cviceni/2006_problem_definitions_and_evaluation_criteria_for_the_cec_2006_special_session_on_constraint_real-parameter_optimization.pdf